Workflow Agentic Automation

Complete Documentation: All 21 Sections

Upstream Prevention · Detailed Diagnostics · 3 Workflows

📚 Documentation Index

1. **14.** Executability Principle
2. **15.** Test Failure Reporting
3. **16.** Assertion Design
4. **17.** Traceability Record
5. **18.** End-to-End Flow (PRD → Automation)
6. **19.** PR Review Prerequisites
7. **20.** Fix PR from Feedback
8. **21.** Environment Confirmation (Dev → STG)

3 Workflows Covered:

1. Create automation from scratch (Engineer)
2. Review PR automation (QA/Lead)
3. Fix PR from feedback (Engineer)

14 — Executability principle: No assumptions

Goal: EVERYTHING must be EXECUTABLE in dev and stg. No aspirational behavior. Every layer verified in real environment before claiming automation.

Core Rule

```
NOT: "Assume the code will work, test later"
BUT: "Verify everything works in REAL environment, then write code"
```

What Must Be Executable

Layer	Must Be Executable	Verify By
SPEC	test.step, expect() runs in real env	Run test in dev/stg
USECASE	Endpoint call works, response valid	Grounding: endpoint exists, responds
Layer 3	Selector found, element exists	Grounding: live inspection of app

Layer	Must Be Executable	Verify By
FACTORY	Generated data insertable	Test: data passes validation, inserts
PRE	Setup endpoint works, DB mutates	Grounding: endpoint exists, verify response
Environment	Dev and stg have SAME structure	Verify: endpoint URL, schema, data

---

Per-Layer Executability Rules

Layer 1: SPEC

✅ **Executable:**

```
test('@covers DF-M1-POS-001', async () => {
  const data = factory.generateEmployee()
  const result = await usecase.execute(data)
  expect(result.status).toBe('DRAFT')  // Actual value, not assumption
})
```

❌ **NOT executable (assumption):**

```
test('Create employee', async () => {
  // Assume usecase.execute works
  const result = await usecase.execute({ name: 'John' })
  expect(result).toBeDefined()  // Assume something will be defined
})
```

Checklist: - [ ] Usecase call = real function (not mock) - [ ] Expected value = from TC (not from code) - [ ] Assertion = meaningful, can fail - [ ] No assumption of success

---

Layer 2: USECASE

✅ **Executable:**

```
export class CreateEmployeeUsecase {
  async execute(data: EmployeeInput): Promise<UsecaseResult> {
    // Call REAL endpoint
    const res = await http.post('/v1/employees', data)

    // Validate response (not assume success)
    if (!res.ok()) {
      throw new Error(`API error: ${res.status()}`)
    }

    return { success: true, data: res.json() }
  }
}
```

❌ **NOT executable (assumption):**

```
export class CreateEmployeeUsecase {
  async execute(data) {
    // Assume endpoint exists (not verified)
    // Assume response has data field
    const res = await http.post('/v1/employees', data)
    return res  // Assume caller will handle error
  }
}
```

Checklist: - [ ] HTTP call = to real endpoint (not mock) - [ ] Response validated = check status, structure - [ ] Error handling = explicit throw (not silent) - [ ] No assumption of success

---

Layer 3: PAGE/SCREEN/COMPONENT

✅ **Executable:**

```
// src: kerjaan-client-portal/src/pages/CreateEmployee.tsx:42
async fillName(name: string) {
  // Selector verified in grounding (live inspection)
  await this.page.fill('[data-testid="employee-name"]', name)

  // Wait for actual network (not fixed sleep)
  await this.page.waitForLoadState('networkidle')
}
```

❌ **NOT executable (assumption):**

```
async fillName(name: string) {
  // Assume selector exists (not verified)
  await this.page.fill('[data-testid="employee-name"]', name)

  // Assume 2 seconds is enough (hardcoded wait)
  await this.page.waitForTimeout(2000)
}
```

Checklist: - [ ] Selector = grounded (live inspected app) - [ ] Selector = commented with // src: - [ ] Wait = dynamic (waitForLoadState, not waitForTimeout) - [ ] No assumption of element existence

---

FACTORY

✅ **Executable:**

```
export function generateEmployee(overrides = {}) {
  // Generate varied, valid data
  return {
    name: faker.person.fullName(),        // Unique per run
    email: faker.internet.email(),        // Valid format
    salary: faker.number.int({ min: 3000000, max: 10000000 }),
    ...overrides
  }
}
```

❌ **NOT executable (assumption):**

```
export function generateEmployee() {
  // Hardcoded same data every run
  return {
    name: 'John Doe',
    email: 'john@test.co',
    salary: 5000000  // Assume this is valid
  }
}
```

Checklist: - [ ] Data = varied (not hardcoded) - [ ] Data = schema-valid (email format, constraints) - [ ] No API call in FACTORY - [ ] No assumption of data uniqueness

---

PRE (Precondition)

✅ Executable:

```
export async function preconditionManagerUser() {
  // PRE endpoint verified in grounding
  const response = await request.post('/v1/admin/seed/user', {
    role: 'manager',
    email: `mgr-${Date.now()}@test.co`  // Unique per run
  })

  // Assert PRE actually worked
  expect(response.status()).toBe(201)
  expect(response.json().id).toBeTruthy()

  return response.json()
}
```

❌ NOT executable (assumption):

```
export async function preconditionManagerUser() {
  // Assume endpoint exists (not verified)
  // Assume it always succeeds
  const response = await request.post('/v1/admin/seed/user', {...})
  return response  // No assertion, assume success
}
```

Checklist: - [ ] PRE endpoint = verified in grounding - [ ] PRE response = validated (status, data) - [ ] PRE data = unique (not reused) - [ ] PRE assertion = explicit (expect status, data)

Environment Parity

Rule: All layers tested in SAME environment (dev/dev or stg/stg).

```
❌ WRONG:
- PRE tested in dev ✓
- Scenario tested in stg ✗
- Endpoint different: dev localhost, stg api.staging.co
- Test fail, unclear why

✅ RIGHT:
- PRE tested in dev
- Scenario tested in dev
- OR both in stg
- Same endpoint, same schema, same behavior
```

Verify:

```
# Check endpoint URL
echo $API_ENDPOINT  # Should be dev or stg, not mixed

# Check environment config
grep -r "localhost\|staging" code/  # Should use ENV, not hardcoded

# Test in target env only
ENVIRONMENT=dev npm test
ENVIRONMENT=stg npm test
# Not mixed!
```

Checklist: Executability per Phase

Phase 1: Grounding

```
[ ] Selector: Live inspection of app, element FOUND
[ ] Endpoint: Search codebase, URL EXISTS
[ ] Expected value: From TC or app response, not memory
[ ] Error scenario: Can this be triggered in real env?
[ ] PRE: Endpoint reachable, response structure known
```

Phase 2: Design

```
[ ] Assertion: Can fail? Meaningful? From TC?
[ ] Error handling: How to detect failure per step?
[ ] PRE sequence: Setup will work in target env?
[ ] Data flow: Data from PRE → scenario → assertion clear?
```

Phase 4: Implement

```
[ ] SPEC: Calls usecase only (not page/API direct)
[ ] USECASE: Calls real endpoint, validates response
[ ] Layer 3: Selector grounded (// src:), no hardcoded
[ ] FACTORY: Data varied, valid schema, not API call
[ ] PRE: Endpoint real, response validated, unique data
```

Phase 5: Verify

```
[ ] PRE: Does setup endpoint respond? DB mutates?
[ ] Test: Does run in target environment? PASSED?
[ ] Selector: Does live inspection confirm element found?
[ ] Response: Does actual response match expected?
[ ] Error: If fail, is error clear (not assumption)?
```

Anti-Patterns: What NOT to Do

> ❌ "The endpoint probably works, let's write test"
> ✅ "Verified endpoint works, now write test"
>
> ❌ "Selector should be there, let me hardcode"
> ✅ "Grounded selector with live inspection, now hardcode"
>
> ❌ "Expected value looks right from code"
> ✅ "Expected value from TC Then clause"
>
> ❌ "Test passed locally, must be correct"
> ✅ "Test passed locally AND in stg, then claim automation"
>
> ❌ "PRE setup probably works"
> ✅ "PRE endpoint responds + DB mutation verified"
>
> ❌ "Generate data same every run"
> ✅ "Generate data varied, unique per run"

Summary: The Principle

> EXECUTABILITY PRINCIPLE:
>
> Every assertion, endpoint, selector, value, and precondition
> must be VERIFIED IN REAL ENVIRONMENT,
> not assumed or hoped.
>
> Code is EXECUTED, not THEORETICAL.
>
> If something cannot be executed in real environment,
> it's not part of the automation – it's blocked.

Reference

- Grounding: `docs/shared/03-codebase-grounding.md`
- Layer architecture: `docs/shared/04-three-layer-architecture.md`
- Test design: `docs/shared/05-test-design-rules.md`
- Assertion design: `docs/shared/16-assertion-design.md`
- Failure reporting: `docs/shared/15-test-failure-reporting.md`

---

15 — Test failure reporting: Detailed error debugging

Goal: Ketika test gagal, error harus dilaporkan dengan detail lengkap dan actionable, bukan generic "test failed". Setiap failure harus traceable ke TC, layer, dan root cause yang jelas.

Rule Utama

1. **Gagal = Detail.** Error message, stack trace, screenshot, video — semuanya harus attached.
2. **Layer identification.** Harus jelas: PRE fail? FACTORY fail? USECASE fail? Layer 3 fail? SPEC assertion fail?
3. **@covers TC-ID required.** Error harus link ke TC yang sedang dijalankan.
4. **Root cause analysis.** "Kenapa gagal?" harus dijawab, bukan diabaikan.
5. **Decision tree.** Kalau gagal, apa action yang harus diambil? Tetap terstruktur.
6. **No silent failures.** Tidak ada exception ditelan, tidak ada tautology, tidak ada assertion semu.

Failure Report Format (Standard)

```
═══════════════════════════════════════════════════════════════
TEST FAILURE REPORT
═══════════════════════════════════════════════════════════════

@covers: [TC-ID] — [TC Title]
Module: [module-name]
Environment: [dev|stg]
Timestamp: [YYYY-MM-DD HH:mm:ss]
Duration: [XX.XXs]

WHICH LAYER FAILED?
  Layer: [SPEC | USECASE | Layer3-PAGE | Layer3-COMPONENT | FACTORY | PRE]
  Severity: [BLOCKER | MAJOR | MINOR]

TEST STEP SEQUENCE:
  ✅ PRE: setup manager user — PASSED (2.3s)
  ✅ SPEC.step('Navigate'): goto('/form') — PASSED (1.1s)
  ✅ SPEC.step('Fill form'): fillName('John') — PASSED (0.8s)
  ❌ SPEC.step('Submit'): clickSubmit() — FAILED (timeout 30s)
  🔲  SPEC.step('Verify'): checkStatus() — NOT EXECUTED

ERROR MESSAGE:
  Timeout 30000ms exceeded.
  Waiting for locator('button:has-text("Submit")').click()

SELECTOR/ENDPOINT INVOLVED:
  button:has-text("Submit")
  Expected to exist: yes
  Actually found: no
  Waited: 30000ms
  Element visible: no

ROOT CAUSE ANALYSIS:
  Layer 3 PAGE.submitForm() — Selector not found in DOM

  Expected selector (from code):
    domain/web/pm/pages/create-employee.page.ts:42
    await page.click('button:has-text("Submit")')
    // src: kerjaan-client-portal/src/pages/CreateEmployee.tsx:52

  Actual selector (from live inspection):
    button:has-text("Save & Continue")
    (button text berbeda!)

  Analysis:
    ❌ Grounding phase found: "Submit"
    ✅ Live DOM shows: "Save & Continue"
```

```
    → Selector outdated atau app changed

DECISION POINT:
  Option A: Fix selector di PAGE layer
    └─ Command: grep -r "Save & Continue" kerjaan-client-portal/src/
    └─ Verify: re-ground selector dengan app terbaru
    └─ Update: pages/create-employee.page.ts:42

  Option B: Re-check grounding
    └─ App berubah sejak grounding phase?
    └─ Re-ground TC scenario terhadap app terbaru

  Option C: Escalate
    └─ UI text mismatch TC expected vs actual
    └─ Decision: TC salah atau app berubah?

EVIDENCE ATTACHED:
  Screenshot: outputs/evidence/2025-01-15/PM-C-001__create-employee__submit-fail
              (Shows DOM state saat selector tidak ketemu)

  Video: outputs/evidence/2025-01-15/PM-C-001__create-employee__submit-fail.webm
         (Shows full flow sampai timeout)

  Browser console:
    Network: No failed requests before timeout
    Console: No JS errors
    Performance: All steps executed normally until submit button

RUN INFORMATION:
  Command: playwright test domain/web/pm/tests/create-employee.spec.ts -g "PM-C-0
  Browser: chromium v130.0.6723.119
  OS: darwin (macOS)
  Node: v20.11.0
  Playwright: 1.48.0

ENVIRONMENT DETAILS:
  URL: https://dev-staffinc.local
  Database: dev postgres (seeded this run)
  Cache: cleared before test
  Network: stable (no timeouts observed elsewhere)

═══════════════════════════════════════════════════════════════
ACTION REQUIRED: [BLOCKED | NEEDS DECISION | FIX REQUIRED]
Assignee: [automation-engineer | QA-lead | developer]
═══════════════════════════════════════════════════════════════
```

Failure Types & Diagnosis

Type 1: PRE (Precondition) Failed

```
@covers: DF-M1-POS-001
WHICH LAYER FAILED? PRE (Precondition Setup)

ERROR:
  POST /v1/admin/seed/user HTTP 500
  Response body: {
    "error": "Database connection timeout",
    "code": "DB_TIMEOUT"
  }

ROOT CAUSE:
  Precondition endpoint tidak responding di environment

  Endpoint: hcmsvc/internal/admin/handler.go:234
  // src: hcmsvc/internal/admin/handler.go:234

  Expected: HTTP 201, users row inserted
  Actual: HTTP 500, no data inserted

DATABASE STATE:
  Before: users table — 42 rows
  After: users table — 42 rows (no insert)
  Conclusion: PRE mutation did NOT happen

DECISION NEEDED:
  ❌ PRE BLOCKED — Cannot proceed to test scenario
  ❌ TC STATUS REMAINS: planned (NOT automated)

  Action items:
    1. Check database connection in [environment]
    2. Check /v1/admin/seed/user endpoint uptime
    3. Retry PRE after environment fixed
    4. Only then try test scenario

NEVER:
  ❌ Skip PRE and "assume" it worked
  ❌ Use hardcoded user ID yang mungkin sudah exist
  ❌ Mark TC as "automated" kalau PRE tidak setup
```

Type 2: FACTORY Data Invalid

```
@covers: DF-M1-POS-002
WHICH LAYER FAILED? FACTORY (Test Data Generation)

ERROR:
  Data validation failed during INSERT:

  INSERT INTO employees (name, email, role)
  VALUES ('John Doe', 'john@test', 'employee')

  Error: Invalid email format 'john@test'
  Constraint: email LIKE '%@%.%'

FACTORY CODE:
  domain/web/pm/factories/employee-data.factory.ts:28

  export function generateEmployee(overrides = {}) {
    return {
      name: faker.person.fullName(),
      email: faker.internet.email(),  ← Generated invalid: "john@test"
      role: 'employee',
      ...overrides
    }
  }

DATA GENERATED (Attempt #1):
  name: "John Doe" ✓
  email: "john@test" ✗ (invalid format)
  role: "employee" ✓

ROOT CAUSE:
  faker.internet.email() occasionally generates invalid format
  (Unlikely but possible with certain seed values)

DECISION:
  Option A: Retry test
    └─ FACTORY failure di 1 attempt = usually one-off
    └─ Run test again, sama FACTORY generate data lagi

  Option B: Enhance FACTORY validation
    └─ Add validator sebelum return data
    └─ Validate email format sebelum INSERT

  Option C: Use static test data
    └─ Replace faker dengan hardcoded valid data
    └─ Trade-off: less variety, more reliability

RECOMMENDATION:
```

```
Retry test dulu (one-off failure kemungkinan besar)
Jika terjadi 2+ kali, enhance FACTORY validator
```

Type 3: USECASE Logic Error

```
@covers: DF-M1-POS-003
WHICH LAYER FAILED? USECASE (Business Logic)

ERROR:
  CreateEmployeeUsecase.execute() returned:
  {
    success: false,
    error: "Validation failed: email already exists"
  }

USECASE CODE:
  domain/web/pm/usecases/create-employee.usecase.ts:15

  async execute(data: EmployeeInput): Promise<UsecaseResult> {
    const existing = await query('SELECT * FROM employees WHERE email=?', data.em
    if (existing.length > 0) {
      return createErrorResult('Validation failed: email already exists')
    }
    const result = await http.post('/v1/employees', data)
    return { success: true, data: result.data }
  }

CALL SEQUENCE:
  ✅ FACTORY.generateEmployee()
     └─ Generated: { name: "John Doe", email: "john@test.co" }

  ❌ USECASE.execute(data)
     └─ Check: email "john@test.co" already exists in DB
     └─ Return: error (did not call HTTP endpoint)

ROOT CAUSE OPTIONS:

  Option A: Database contamination
    └─ DB tidak dibersihkan antara test run
    └─ Email "john@test.co" dari test sebelumnya masih exist
    └─ Solution: Clean DB before each test run

  Option B: FACTORY reuse
    └─ Same FACTORY instance reused across tests
    └─ Solution: Generate fresh data per test

  Option C: USECASE validation wrong
    └─ Validation rule tidak sesuai TC expected behavior
    └─ Solution: Review TC, check if "email exists" check is expected

DECISION TREE:
  1. Check: TC expected behavior apa?
```

```
     └─ TC bilang: "Create dengan email unik" → OK (error is expected)
     └─ TC bilang: "Create langsung dengan email any" → FAIL (validation wrong)

  2. Check: Database state
     └─ SELECT COUNT(*) FROM employees WHERE email='john@test.co'
     └─ If > 0: Database contamination
     └─ If = 0: Something else wrong

  3. Check: FACTORY consistency
     └─ Every call generateEmployee() should return unique email
     └─ If not: FACTORY returns same email

NEXT STEPS:
  If validation expected: Mark test as PASSED (error is correct)
  If validation unexpected: Fix USECASE or DB cleanup
```

Type 4: Layer 3 (PAGE/COMPONENT) Failed

```
@covers: DF-M1-POS-004
WHICH LAYER FAILED? Layer 3 — PAGE (UI Interaction)

ERROR:
  Timeout 30000ms exceeded.
  page.fill('[data-testid="employee-name"]')

  Element: [data-testid="employee-name"]
  Status: NOT FOUND

PAGE CODE:
  domain/web/pm/pages/create-employee.page.ts:42

  async fillName(name: string) {
    await page.fill('[data-testid="employee-name"]', name)
  }
  // src: kerjaan-client-portal/src/pages/CreateEmployee.tsx:52

GROUNDING EVIDENCE:
  Grounding phase found:
    ✅ Selector: [data-testid="employee-name"]
    ✅ Located: kerjaan-client-portal/src/pages/CreateEmployee.tsx:52
    ✅ Element visible in live inspection

  Current app state:
    ❌ Selector: [data-testid="employee-name"] — NOT FOUND
    ✅ Selector: [data-testid="first-name"] — FOUND

  Analysis:
    Grounding found "employee-name"
    Live app has "first-name"
    → Selector outdated OR app changed since grounding

ROOT CAUSE:

  Sub-cause A: Selector in code is wrong
    └─ PAGE layer has typo or old selector
    └─ Fix: Update PAGE layer with correct selector

  Sub-cause B: App UI changed
    └─ data-testid attribute changed in app
    └─ Fix: Re-ground selector against current app version

  Sub-cause C: Grounding was wrong
    └─ Original grounding found wrong selector
    └─ Fix: Re-ground TC scenario from scratch
```

```
HTTP INTERACTION (if applicable):
  Before timeout, network calls:
    ✅ GET /form - 200 OK (page loaded)
    ✅ GET /form/fields - 200 OK (schema loaded)
    ❌ After: No more requests (stuck at fill)

DOM INSPECTION (screenshot):
  Live DOM at failure time:
    <form>
      <input data-testid="first-name" />
      <input data-testid="last-name" />
      <input data-testid="email" />
    </form>

  Expected element: [data-testid="employee-name"]
  Found element: [data-testid="first-name"]

DECISION:
  Option A: Fix PAGE layer selector
    └─ Update: [data-testid="employee-name"]
    └─ To: [data-testid="first-name"]
    └─ Verify: Re-run test

  Option B: Re-ground selector
    └─ Run grounding phase again
    └─ Update grounding notes
    └─ Verify selector actually exists in current app

  Option C: Verify TC matches app
    └─ TC says: "Fill employee name field"
    └─ App shows: "Fill first name field"
    └─ Decision: TC or app wrong?
```

Type 5: SPEC Assertion Failed

```
@covers: DF-M1-POS-005
WHICH LAYER FAILED? SPEC (Assertion)

ASSERTION THAT FAILED:
  expect(result.status).toBe('DRAFT')

  Expected: 'DRAFT'
  Actual: 'PENDING'

SPEC CODE:
  domain/web/pm/tests/create-employee.spec.ts:52

  test('@covers DF-M1-POS-005', async () => {
    const result = await createEmployeeUsecase.execute(data)
    expect(result.status).toBe('DRAFT')  ← ASSERTION FAILED
  })

TC EXPECTED RESULT:
  From staffinc-testcase/testcases/web/pm/DF-M1-POS-005.md:

  "Then: Employees harus dibuat dengan status DRAFT"

ACTUAL BEHAVIOR:
  When POST /v1/employees called:
    Response: { id: 123, name: "John", status: 'PENDING' }

  Database: employees table, id=123, status='PENDING'

MISMATCH ANALYSIS:

  TC says: Status DRAFT after create
  Code does: Status PENDING after create

  Question: Kenapa status PENDING, bukan DRAFT?

  Option A: App workflow changed
    └─ Employees now auto-transition to PENDING on create
    └─ This is a DEFECT (app behavior ≠ TC requirement)
    └─ Action: Report defect, update TC, or fix app

  Option B: TC is outdated
    └─ Requirement changed but TC not updated
    └─ Action: Update TC expected result
    └─ But: Verify with Product/QA first

  Option C: Assertion is wrong
    └─ Maybe code is correct, assertion misread
```

```
    └─ But: Unlikely if assertion directly from TC

DECISION TREE (critical):

  🚫 WRONG ACTION:
     ❌ Change assertion to: expect(result.status).toBe('PENDING')
     ❌ Use loose matcher: expect(result.status).toBeTruthy()
     ❌ Comment out assertion
     ❌ Ignore mismatch and mark as "passed"

     ALL OF ABOVE = forgery, test is not valid

  ✅ RIGHT ACTION:
     1. Verify TC: Open staffinc-testcase/testcases/web/pm/DF-M1-POS-005.md
        └─ Check: Expected Result says DRAFT? Confirms

     2. Check codebase: Why is status PENDING?
        └─ Search: hcmsvc/internal/employee/handler.go
        └─ Find: POST /v1/employees endpoint
        └─ Check: Default status = PENDING?
        └─ Find: Is there workflow auto-transition?

     3. Decide with team:
        A) App is correct: Update TC expected result
           └─ Edit: staffinc-testcase/testcases/web/pm/DF-M1-POS-005.md
           └─ Change: Expected Result → "status PENDING"
           └─ Reason: App behavior changed, TC must reflect reality

        B) TC is correct: App has defect
           └─ Create: defect ticket
           └─ Description: "Create employee returns PENDING, should be DRAFT"
           └─ Reference: TC DF-M1-POS-005
           └─ Action: Fix app, then test passes

        C) Confusion on requirement:
           └─ Escalate to Product/QA
           └─ Clarify: What should status be after create?
           └─ Update: TC based on clarification

NEVER (repeat: NEVER):
  ❌ "Let me just change assertion to PENDING"
  ❌ "The test passes now"
  ❌ This is AUTOMATION FORGERY
  ❌ Root cause: App behavior ≠ TC requirement
  ❌ Hiding it = covering up defect
```

Decision Tree: What to Do When Test Fails?

```
TEST FAILED
    ↓
╔═══════════════════════════════════════════════════════════════╗
║ STEP 1: Identify which LAYER failed                           ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  A) PRE (Precondition)                                        ║
║     └─ Setup failed, test scenario cannot run                 ║
║     └─ Check: /v1/admin/seed/user endpoint                    ║
║     └─ Check: Database state after PRE                        ║
║     └─ Action: Fix environment, then RETRY                    ║
║     └─ TC Status: remains PLANNED (not automated)             ║
║                                                               ║
║  B) FACTORY (Data Generation)                                 ║
║     └─ Generated data invalid                                 ║
║     └─ Check: Data matches schema?                            ║
║     └─ Check: Data unique (no conflicts)?                     ║
║     └─ Action: RETRY (usually one-off)                        ║
║                └─ If repeats: enhance FACTORY validation      ║
║                                                               ║
║  C) USECASE (Business Logic)                                  ║
║     └─ Logic returned error                                   ║
║     └─ Check: Error expected per TC?                          ║
║     └─ Check: USECASE logic matches TC?                       ║
║     └─ Check: Input data valid?                               ║
║     └─ Action: FIX USECASE or DB cleanup                      ║
║                                                               ║
║  D) Layer 3 (PAGE/COMPONENT/Endpoint)                         ║
║     └─ Selector not found OR endpoint 404/500                 ║
║     └─ Check: Selector exists in current app?                 ║
║     └─ Check: Endpoint still exists?                          ║
║     └─ Check: Grounding outdated?                             ║
║     └─ Action: RE-GROUND selector/endpoint                    ║
║                └─ Or: UPDATE Layer 3 code                     ║
║                                                               ║
║  E) SPEC (Assertion)                                          ║
║     └─ Expected value ≠ Actual value                          ║
║     └─ Check: TC expected result accurate?                    ║
║     └─ Check: App behavior matches TC?                        ║
║     └─ Decision: UPDATE TC or REPORT DEFECT                   ║
║     └─ 🚫 NEVER adjust assertion to pass                      ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
    ↓
╔═══════════════════════════════════════════════════════════════╗
║ STEP 2: Gather evidence                                       ║
╠═══════════════════════════════════════════════════════════════╣
```

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║  Screenshot: outputs/evidence/[YYYY-MM-DD]/[TC-ID].png        ║
║  Video: outputs/evidence/[YYYY-MM-DD]/[TC-ID].webm            ║
║  Console logs: Browser errors                                 ║
║  Network logs: HTTP requests/responses                        ║
║  Stack trace: Full error stack                                ║
║  Command: Exact command that was run                          ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝

    ↓

╔═══════════════════════════════════════════════════════════════╗
║ STEP 3: Root cause analysis                                   ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  ❓ What exactly failed?                                      ║
║     └─ Not "test failed", but "Layer X → reason Y"            ║
║                                                               ║
║  ❓ Why did it fail?                                          ║
║     └─ Selector wrong? Endpoint not responding? Data invalid? ║
║                                                               ║
║  ❓ Is failure expected per TC?                               ║
║     └─ If yes: Assertion should verify it                     ║
║     └─ If no: Something is broken                             ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝

    ↓

╔═══════════════════════════════════════════════════════════════╗
║ STEP 4: Decision & Action                                     ║
╠═══════════════════════════════════════════════════════════════╣
║                                                               ║
║  BLOCKER (do not pass):                                       ║
║    ❌ PRE setup failed – environment issue                    ║
║    ❌ SPEC assertion fail (app ≠ TC) – needs decision         ║
║    ❌ Layer 3 selector wrong – needs fix                      ║
║    ❌ @covers TC-ID missing – spec issue                      ║
║                                                               ║
║  ACTIONABLE (fix and retry):                                  ║
║    ✅ FACTORY data invalid – enhance FACTORY                  ║
║    ✅ USECASE error (expected) – fix if TC changed            ║
║    ✅ Layer 3 outdated – re-ground and update                 ║
║                                                               ║
║  DECISION NEEDED (escalate):                                  ║
║    ⚠️  App behavior ≠ TC expected – product decision          ║
║    ⚠️  Requirement unclear – QA lead decision                 ║
║    ⚠️  Environment down – ops decision                        ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
```

Evidence Requirements

Ketika test gagal, HARUS attach evidence:

Screenshot

```
Lokasi: outputs/evidence/[YYYY-MM-DD]/
Nama: [TC-ID]__[spec-name]__[failure-point].png

Contoh: PM-C-001__create-employee__selector-not-found.png

Requirement:
  ✅ Show DOM state saat gagal
  ✅ Show error message (kalau ada di UI)
  ✅ Include browser dev tools console (errors visible)
  ✅ Timestamp visible (metadata bar)
  ✅ Full viewport (tidak cropped)
```

Video

```
Lokasi: outputs/evidence/[YYYY-MM-DD]/
Nama: [TC-ID]__[spec-name]__[scenario].webm

Contoh: PM-C-001__create-employee__full-flow.webm

Requirement:
  ✅ Full test execution (PRE → scenario → failure)
  ✅ Show exact moment of failure (pause frame)
  ✅ Include network tab (HTTP calls visible)
  ✅ Duration < 5 minutes (reasonable length)
  ✅ Format: webm or mp4
```

Logs

```
Browser Console:
  - All errors printed
  - All network errors
  - All warnings related to test
  - Example:
    Error: fill: element not found <selector>
    at page.fill() [createEmployee.page.ts:42]

Network Log:
  - All HTTP requests
  - Response status + headers
  - Response body (JSON/error)
  - Timing info
  - Example:
    POST /v1/employees HTTP 500
    Response: { error: "Validation failed" }
    Timing: 234ms

Stack Trace:
  - Full stack (bukan truncated)
  - Include file paths + line numbers
  - Include layer info (Layer3 / USECASE / SPEC)
```

Run Information

```
Command:
  playwright test domain/web/pm/tests/create-employee.spec.ts -g "PM-C-001"

Output:
  1 failed (42.3s)
  [chromium] › domain/web/pm/tests/create-employee.spec.ts:42:3
  › 01  [UI] Create employee with valid data

  Error: Timeout 30000ms exceeded.
```

Reporting to Team/Channel

❌ BAD Report:

```
"Test PM-001 gagal"
"Ada error di create employee"
"Selector tidak ketemu, kelihatannya app berubah"
"Udah coba diulangi tapi tetap gagal"
```

✅ GOOD Report:

```
TEST FAILURE: DF-M1-POS-001

@covers: DF-M1-POS-001 — Create employee with valid data
Layer: Layer 3 — PAGE.fillName()
Error: Timeout 30000ms | selector [data-testid="employee-name"] not found

ROOT CAUSE:
  Grounding found: [data-testid="employee-name"]
  Live app shows: [data-testid="first-name"]
  Diagnosis: Selector outdated OR app changed

EVIDENCE:
  Screenshot: outputs/evidence/2025-01-15/DF-M1-POS-001.png
  Video: outputs/evidence/2025-01-15/DF-M1-POS-001.webm
  Command: playwright test domain/web/pm/tests/create-employee.spec.ts -g "DF-M1

ACTION NEEDED:
  Option A: Fix PAGE layer → update selector to [data-testid="first-name"]
  Option B: Re-ground → verify selector against current app
  Option C: Check TC → does TC describe "employee name" or "first name"?

ASSIGNMENT:
  Automation engineer: Update selector if app UI finalized
  QA lead: Verify TC accuracy
  Developer: Confirm UI is intentional change
```

Checklist: Sebelum Report "Test Failed"

```
REQUIRED:
  [ ] @covers TC-ID ada dan benar?
  [ ] Tahu layer mana yang fail? (PRE/FACTORY/USECASE/Layer3/SPEC)
  [ ] Error message lengkap (bukan truncated)?
  [ ] Evidence attached (screenshot/video)?
  [ ] Root cause analyzed (bukan "dunno")?
  [ ] Decision path clear (action item defined)?
  [ ] Command yang dijalankan dicatat?

RED FLAGS (do not pass):
  [ ] "Assertion agar pass" — NEVER
  [ ] "Silent failure" — NEVER
  [ ] "Exception ditelan" — NEVER
  [ ] "Tautology assertion" — expect(true).toBe(true)
  [ ] "@covers missing" — BLOCKER
  [ ] "Tidak ada evidence" — BLOCKER
  [ ] "Root cause tidak jelas" — BLOCKER
```

Anti-Patterns: What NOT to Do

❌ Silent Failures

```
WRONG:
  test('Create employee', async () => {
    try {
      await page.fill('[data-testid="name"]', 'John')
    } catch (e) {
      console.log(e)  // Error logged but test continues
    }
    expect(true).toBe(true)  // Always passes
  })

Result: Test passes even though selector failed

RIGHT:
  test('Create employee', async () => {
    await page.fill('[data-testid="name"]', 'John')
    // No try/catch - let error propagate
    expect(page.locator('[data-testid="name"]')).toHaveValue('John')
    // Explicit assertion
  })
```

❌ Tautology Assertions

```
WRONG:
  expect(result).toBeDefined()  // Always true if reached
  expect(result.id > 0).toBeTruthy()  // True for any positive number
  expect(true).toBe(true)  // Always passes

RIGHT:
  expect(result.status).toBe('DRAFT')  // Can fail if status is PENDING
  expect(result.id).toBe(expectedId)  // Specific value check
  expect(employees).toHaveLength(expectedCount)  // Countable
```

❌ Assertion Forgery

```
WRONG:
  // TC says: status DRAFT
  // App returns: status PENDING
  // Solution: change assertion

  expect(result.status).toBe('PENDING')  ← FORGERY
  // Now test passes, but TC requirement is violated

RIGHT:
  // Keep assertion as per TC
  expect(result.status).toBe('DRAFT')

  // If test fails: decision point
  - Is TC wrong? Update TC (with approval)
  - Is app wrong? Report defect (fix app)
  - Never silently change assertion
```

❌ Hidden Dependencies

```
WRONG:
  test('Create', async () => {
    const result = await create(data)  // Assume success
    expect(result.id).toBeTruthy()  // But what if create failed?
  })

Result: If create returns error, assertion misleading

RIGHT:
  test('Create', async () => {
    const result = await create(data)
    expect(result.success).toBe(true)  // Verify create succeeded
    expect(result.id).toBeTruthy()  // Then verify ID exists
  })
```

Summary: Executability Principle + Debuggability

```
Automation testing prinsip:

1. EXECUTABLE
   └─ Test harus bisa run di dev/stg
   └─ Tidak ada assumption, semua verified

2. DEBUGGABLE
   └─ Kalau fail, error jelas dan actionable
   └─ Harus tahu: layer mana? kenapa? apa action-nya?

3. TRACEABLE
   └─ Error → @covers TC → root cause
   └─ Evidence lengkap (screenshot/video)

4. ACCOUNTABLE
   └─ Tidak ada silent failures
   └─ Tidak ada assertion forgery
   └─ Tidak ada "magic" atau assumption
```

Next Document

Dokumentasi ini mengasumsikan test sudah written dan dijalankan. Untuk error handling di **grounding phase** dan **layer design phase**, lihat: - `docs/shared/03-codebase-grounding.md` — ground dulu sebelum code - `docs/shared/04-three-layer-architecture.md` — DDD rules per layer - `workflow/01-grounding.md` — grounding phase detail

---

16 — Assertion design & error handling

Goal: Design assertion and error handling from TC BEFORE writing code. Prevent tautology and swallowed errors upstream.

Rule 1: Every Assertion Must Be Able to FAIL

```
❌ TAUTOLOGY (always pass):
  expect(true).toBe(true)
  expect(result).toBeDefined()
  expect(count).toBeGreaterThanOrEqual(0)

✅ MEANINGFUL (can fail):
  expect(result.status).toBe('DRAFT')
  expect(employees).toHaveLength(3)
  expect(form).toBeVisible()
```

Rule 2: Every Error Must PROPAGATE

```
❌ SWALLOWED (error hidden):
  .catch(() => false)
  .catch(() => {})
  try { } catch(e) { console.log(e) }
  result.success || true

✅ PROPAGATE (error clear):
  // Let error throw naturally
  throw new Error(`Failed: ${message}`)
  if (!response.ok()) throw new Error(...)
```

Rule 3: Every Value from TC (not code)

```
❌ HARDCODED (from memory):
  expect(result.status).toBe('ACTIVE')  // code assumes
  const name = 'John'  // hardcoded

✅ FROM TC (traceable):
  expect(result.status).toBe('DRAFT')  // from TC Then#1
  const data = factory.generateEmployee()  // varied, from FACTORY
```

Assertion Design Template (Grounding Phase)

For each TC Then/And clause:

TC Clause	Observable	Expected	Assertion	Can Fail?
		"DRAFT"	expect(...).toBe('DRAFT')	

TC Clause	Observable	Expected	Assertion	Can Fail?
"Status DRAFT"	result.status field			✅ Yes
"Row exists"	row in table	row.id === X	expect(rows).toContainEqual({id: X})	✅ Yes
"Form visible"	form element	visible	expect(form).toBeVisible()	✅ Yes

---

Error Handling Pattern (per Layer)

SPEC Layer

```
test('@covers DF-M1-POS-001', async () => {
  // Let error propagate (not swallow)
  const result = await usecase.execute(data)

  // Assertion: can fail if feature broken
  expect(result.status).toBe('DRAFT')
})
```

USECASE Layer

```
export class CreateEmployeeUsecase {
  async execute(data: EmployeeInput): Promise<UsecaseResult> {
    try {
      const res = await http.post('/v1/employees', data)
      if (!res.ok()) {
        throw new Error(`API error: ${res.status()}`)
      }
      return { success: true, data: res.json() }
    } catch (e) {
      // Re-throw with context (not swallow)
      throw new Error(`Create failed: ${e.message}`)
    }
  }
}
```

Layer 3 (PAGE)

```
async fillName(name: string) {
  // Let error propagate (not .catch)
  await this.page.fill('[data-testid="name"]', name)

  // Positive anchor (not negative read)
  await expect(this.page.locator('[data-testid="name"]')).toHaveValue(name)
}
```

FACTORY

```
export function generateEmployee(overrides = {}) {
  // Validate data (not assume valid)
  const data = {
    name: faker.person.fullName(),
    email: faker.internet.email(),
    ...overrides
  }

  // Optional: validate schema
  if (!data.email.includes('@')) {
    throw new Error('Invalid email generated')
  }

  return data
}
```

PRE (Precondition)

```
export async function preconditionManagerUser() {
  const response = await request.post('/v1/admin/seed/user', {
    role: 'manager',
    email: `mgr-${Date.now()}@test.co`
  })

  // Assert PRE executed (not assume)
  expect(response.status()).toBe(201)
  expect(response.json().id).toBeTruthy()

  return response.json()
}
```

Checklist: Before Implementing

```
Assertion Design:
  [ ] No tautology (not expect(true).toBe(true))
  [ ] Can fail (jika feature broken)
  [ ] From TC expected, not code
  [ ] // expect: <TC-ID> Then#n comment added
  [ ] One assertion per TC clause

Error Handling:
  [ ] No .catch(() => false)
  [ ] No try/catch hiding error
  [ ] Errors propagate with message
  [ ] PRE assertion explicit
  [ ] USECASE error handling clear

Data & Values:
  [ ] No hardcoded data (use FACTORY)
  [ ] Dynamic email/id per run
  [ ] Expected value from TC, not memory
  [ ] Selector from grounding (// src:)

DDD:
  [ ] SPEC calls usecase only
  [ ] USECASE no expect()
  [ ] Layer 3 no business logic
  [ ] FACTORY no API calls
```

Red Flags During Code Review

```
BLOCKER:
  ❌ Tautology: expect(true), toBeDefined(), toBeGreaterThanOrEqual(0)
  ❌ Swallowed error: .catch(() => false)
  ❌ Hardcoded value: 'John Doe', 'john@test.co'
  ❌ Missing assertion: TC clause with no expect()
  ❌ DDD broken: SPEC touch page, USECASE with expect()
  ❌ No error handling: try/catch with console.log
```

Reference

- Executability: `docs/shared/14-executability-principle.md`
- Test design: `docs/shared/05-test-design-rules.md`
- Layer architecture: `docs/shared/04-three-layer-architecture.md`
- Failure reporting: `docs/shared/15-test-failure-reporting.md`

---

17 — Traceability record: Automation to TC

Goal: Every automation must be traceable back to canonical TC. Record verification in centralized checklist.

What to Record

1. AUTOMATION-CHECKLIST-.md (Centralized)

Location: `domain/[web|mobile|api]/[module]/AUTOMATION-CHECKLIST-<TC-ID>.md`

Example: `domain/web/pm/AUTOMATION-CHECKLIST-DF-M1-POS-001.md`

Template: AUTOMATION-CHECKLIST

```
# Automation Checklist: <TC-ID>

TC: <TC-ID> – <TC Title>
Automation engineer: [name]
Date: [YYYY-MM-DD]

## Given (Preconditions)

- [ ] PRE: [what precondition]
  - Factory: `preconditionXxx()`
  - Location: [file:line]
  - Endpoint: [POST /v1/...]
  - Status code: 201 ✓
  - DB mutation verified: ✓

## When (Actions)

- [ ] Action: [navigate/fill/click/submit]
  - Code: [file:line]
  - Selector: [data-testid="..."]
  - Grounded: // src: [codebase/file:line] ✓
  - Can fail: YES ✓

## Then/And (Expected Results)

### Then#1: [TC clause]
- [ ] Assertion: `expect(...).toBe(...)`
  - Location: [file:line]
  - Comment: // expect: <TC-ID> Then#1
  - Can fail: YES ✓
  - From TC: YES ✓
  - Tautology: NO ✓

### And#1: [TC clause]
- [ ] Assertion: `expect(...).toBe(...)`
  - Location: [file:line]
  - Comment: // expect: <TC-ID> And#1
  - Can fail: YES ✓
  - From TC: YES ✓

## Red Flag Prevention

- [ ] Tautology: NO ✓
- [ ] Swallowed error: NO ✓
- [ ] Hardcoded value: NO ✓
- [ ] DDD broken: NO ✓
```

```
## Summary

- Total TC clauses: [N]
- Assertions implemented: [N/N] ✓
- Coverage: 100% ✓
- Status: READY FOR PR ✓
```

---

2. Code Comments (Distributed)

Scattered dalam SPEC/USECASE/PAGE/FACTORY:

```
// @covers <TC-ID>
test('...', async () => {
  // // expect: <TC-ID> Then#1
  expect(...).toBe(...)
})

// src: filepath:line (selector)
await page.fill('[data-testid="..."]', value)
```

---

3. Pre-push Checklist

In `docs/shared/11-pre-push-review.md`:

```
[ ] AUTOMATION-CHECKLIST-<TC-ID>.md exist?
[ ] All TC clauses covered?
[ ] Code comments (@covers, // expect:, // src:) present?
[ ] Red flags scanned (tautology, swallowed error)?
[ ] Test result: PASSED?
```

Verification Process

Phase 4: Implement

> 1. Write code (SPEC/USECASE/Layer3/FACTORY)
> 2. Add comments (@covers, // expect:, // src:)
> 3. Create AUTOMATION-CHECKLIST-<TC-ID>.md
> 4. Fill checklist: every TC clause → assertion mapping
> 5. Self-verify: can fail? meaningful? from TC?
> 6. Self-sign: engineer mark ✓

Phase 5: Verify

> 1. Run test → PASSED (or debug + fix)
> 2. Verify PRE executed ✓
> 3. Collect evidence (video, logs)
> 4. Check AUTOMATION-CHECKLIST complete ✓

Phase 6: PR Review

> 1. Read AUTOMATION-CHECKLIST
> 2. Confirm: verification done upstream ✓
> 3. No blocker: APPROVED
> 4. Merge → Automation status = true

Red Flags in Checklist

If any of these unchecked = REQUEST_CHANGES:

❌ Total TC clauses ≠ Assertions implemented
→ Missing assertion for some clause

❌ Tautology: YES
→ Fix: make assertion meaningful

❌ Swallowed error: YES
→ Fix: let error propagate

❌ Hardcoded value: YES
→ Fix: use FACTORY or from TC

❌ Can fail: NO
→ Fix: assertion must be able to fail

❌ From TC: NO
→ Fix: expected value must be from TC

❌ Test result: FAILED or SKIPPED
→ Fix: run test, get PASSED only

Sample AUTOMATION-CHECKLIST

```
# Automation Checklist: DF-M1-POS-001

TC: DF-M1-POS-001 — Create employee with valid data
Automation engineer: [name]
Date: 2026-01-15

## Given (Preconditions)

- [x] PRE: Manager user exists
  - Factory: `preconditionManagerUser()`
  - Location: domain/web/pm/factories/preconditions.factory.ts:23
  - Endpoint: POST /v1/admin/seed/user
  - Status code: 201 ✓
  - DB mutation verified: users.role = 'manager' ✓

## When (Actions)

- [x] Navigate to form
  - Code: domain/web/pm/tests/create-employee.spec.ts:15
  - Selector: [data-testid="create-form"]
  - Grounded: // src: kerjaan-client-portal/src/pages/.../CreateEmployee.tsx:42 ✓

- [x] Fill name
  - Code: domain/web/pm/tests/create-employee.spec.ts:18
  - Selector: [data-testid="employee-name"]
  - Data: 'John Smith' (from TC Given) ✓

- [x] Submit
  - Code: domain/web/pm/tests/create-employee.spec.ts:21
  - Wait: networkidle ✓

## Then/And (Expected Results)

### Then#1: "Status DRAFT"
- [x] Assertion: `expect(result.status).toBe('DRAFT')`
  - Location: domain/web/pm/tests/create-employee.spec.ts:26
  - Comment: // expect: DF-M1-POS-001 Then#1 ✓
  - Can fail: YES ✓
  - From TC: YES ✓
  - Tautology: NO ✓

### Then#2: "Employee ID returned"
- [x] Assertion: `expect(result.id).toBeTruthy()`
  - Location: domain/web/pm/tests/create-employee.spec.ts:27
  - Comment: // expect: DF-M1-POS-001 Then#2 ✓
  - Can fail: YES ✓
```

```
## Red Flag Prevention

- [x] Tautology: NO ✓
- [x] Swallowed error: NO ✓
- [x] Hardcoded value: NO ✓
- [x] DDD broken: NO ✓

## Summary

- Total TC clauses: 2 (2 Then)
- Assertions implemented: 2/2 ✓
- Coverage: 100% ✓
- Status: READY FOR PR ✓

Self-reviewed by: [engineer name]
Date: 2026-01-15 ✓
```

Reference

- Assertion design: `docs/shared/16-assertion-design.md`
- Pre-push review: `docs/shared/11-pre-push-review.md`
- Implementation: `workflow/04-implement.md`
- Failure reporting: `docs/shared/15-test-failure-reporting.md`

---

18 — End-to-End Flow: PRD → Automation Live

Goal: Complete flow from PRD to automated test running on stg/dev with upstream prevention at each gate.

🎯 Overview: 3 GATES + 6 PHASES

```
INPUT: PRD (Product Requirements Document)
       ↓
┌──────────────────────────────────────┐
| GATE A: TSD-writer                   |
| Convert PRD → TSD (validated)        |
| Tool: repos/staffinc/skills/tsd-writer
| Output: TSD file + self-check        |
└──────────────────────────────────────┘
       ↓
┌──────────────────────────────────────┐
| GATE B: testcase-writer              |
| Convert TSD → TC (canonical)         |
| Tool: repos/staffinc/skills/testcase-writer
| Output: TC + PRE files               |
└──────────────────────────────────────┘
       ↓
┌──────────────────────────────────────┐
| GATE C: Automation Workflow          |
| Phase 0-6: Grounding → Live          |
| Tool: Workflow phases                |
| Output: Automated test live          |
└──────────────────────────────────────┘
       ↓
OUTCOME: TC marked automated, test running on stg/dev
```

GATE A: TSD-writer (PRD → TSD)

Input: PRD document

Tool: `skills/tsd-writer/SKILL.md`

Output: TSD file (validated) + self-check results

Steps I Will Do:

1. Read PRD Completely

```
├─ Requirements: What user can/should do?
├─ Acceptance Criteria: How to verify feature works?
├─ Boundary conditions: Edge cases, limits?
├─ Dependencies: Other features, permissions, data?
├─ Non-functional: Performance, security, UI/UX?
└─ Constraints: Budget, timeline, technical limits?
```

2. Create TSD File

Location: `domain/[web|mobile|api]/[module]/TSD-<FEATURE>.md`

Example: `domain/web/pm/TSD-CREATE-EMPLOYEE.md`

Content sections:

```
# TSD: Create Employee

## 1. Feature Title & Description
- From PRD: "Manager dapat membuat employee baru"
- Scope: Web platform only, manager role
- Out of scope: API, mobile, employee edit

## 2. Acceptance Criteria (from PRD)
- AC#1: Form tersedia untuk manager
- AC#2: Valid data accepted, record created
- AC#3: Invalid data rejected with error
- AC#4: Notification sent to HR

## 3. Preconditions (what must be true)
- PRE#1: Manager user exists
- PRE#2: Test environment ready (stg/dev)
- PRE#3: Email service available

## 4. Test Scenarios

### POS-001: Create with valid data
- **Given:** Manager logged in, form open
- **When:** Fill name, email, department
- **Then:** Record created, notification sent
- **Data:** From FACTORY (not hardcoded)

### NEG-001: Create with invalid email
- **Given:** Manager logged in, form open
- **When:** Fill invalid email format
- **Then:** Error shown, record NOT created

### EDG-001: Duplicate email
- **Given:** Manager, existing email in DB
- **When:** Try create with same email
- **Then:** Error: "Email exists"

## 5. Dependencies
- Feature X (must be done first)
- Permission API (endpoint must be stable)
- Test data: Employee schema finalized

## 6. Data & State
- Test email: faker.person.email() (varied)
- Test name: faker.person.fullName() (varied)
- State: Fresh DB per test (cleanup)
```

3. Self-Check (TSD Quality)

Before handoff, verify:

```
[ ] 12 TSD sections present?
[ ] AC extracted from PRD (not invented)?
[ ] Scope: Included/excluded clear?
[ ] Scenarios: POS/NEG/EDG covered?
[ ] PRE: What setup needed? (data, state, permissions)
[ ] Data: Varied (FACTORY), not hardcoded?
[ ] Given/When/Then: Observable, testable, clear?
[ ] References: PRD cited with page/section?
[ ] Count: N scenarios, N AC, N PRE
[ ] Status: DRAFT or VALIDATED?
```

Output: TSD file + self-check checklist

GATE B: testcase-writer (TSD → TC)

Input: Validated TSD

Tool: `skills/testcase-writer/SKILL.md`

Output: TC files + PRE files in `staffinc-testcase` repo

Steps I Will Do:

1. Read TSD + TC Schema

- TSD from GATE A
- TC contract: `staffinc-testcase/SCHEMA.md`
- PRE contract: `staffinc-testcase/PRECONDITIONS.md`

2. Create TC File per Scenario

Location: `testcase/[domain]/[module]/TC-<FEATURE>-<SCENARIO>.md`

Example: `testcase/web/pm/TC-CREATE-EMPLOYEE-POS-001.md`

Content:

> # Test Case: DF-M1-POS-001
>
> **Title:** Create employee with valid data (from TSD POS-001)
>
> **TSD Reference:** domain/web/pm/TSD-CREATE-EMPLOYEE.md#POS-001
>
> ## Given (Preconditions)
>
> What must be true before user action:

1. Manager user exists (email: mgr-xxx@test.co)
2. Factory: preconditionManagerUser()
3. Endpoint: POST /v1/admin/seed/user
4. Expected status: 201
5. DB: users.role = 'manager' verified
6. Executable: YES (dev/stg)
7. Form endpoint available
8. Endpoint: GET /v1/pm/create-form
9. Expected status: 200
10. Executable: YES (dev/stg)

> ## When (User Actions)
>
> Observable sequence of actions:

1. Navigate to create employee form
2. URL: /pm/employees/create
3. Observable: Form element visible
4. Selector: [data-testid="create-form"]
5. Fill name field
6. Action: page.fill('[data-testid="name"]', name)
7. Data: 'John Smith' (from FACTORY, varied)

8. Observable: Input value = 'John Smith'
9. Fill email field
10. Action: page.fill('[data-testid="email"]', email)
11. Data: faker.internet.email() (varied per run)
12. Observable: Input value = email
13. Click submit button
14. Action: page.click('[data-testid="submit"]')
15. Wait: network idle, response 201
16. Verify notification sent
17. Observable: Toast message appears
18. Text: "Employee created successfully"

```
## Then/And (Expected Results)

What should be true after actions:
```

Then#1: Record Created

- API Response: status 201
- Response body: { id: , status: 'DRAFT' }
- Observable: Record in DB with correct data

Then#2: Status is DRAFT

- Assertion: result.status === 'DRAFT'
- From TC: ✅ (TSD specifies)
- Can fail: ✅ (if code sets ACTIVE)

And#1: Notification Sent

- Email sent: YES
- Recipient: HR team email
- Content: "New employee: John Smith"

> ## Preconditions (PRE) Reconciliation

PRE#1: Manager user exists Status: REUSE existing precondition File: domain/web/pm/factories/preconditions.factory.ts Function: preconditionManagerUser()
PRE#2: Form endpoint available Status: NEW precondition needed File: domain/web/pm/factories/preconditions.factory.ts Function: preconditionFormAvailable() [TO BE CREATED] Action: GET /v1/pm/create-form, verify 200

> ## Execution Environment

Environments where this TC must be executable: - dev: YES (full control, reset DB) - stg: YES (test data available) - prod: NO (read-only, no creation)
Data setup: - Test account: manager-xxx@test.co (varied per run) - Test data: From FACTORY (not hardcoded) - Cleanup: Delete created record after test

> ## Coverage Summary

TC ID: DF-M1-POS-001 TSD Scenario: POS-001 (Create with valid data) AC covered: AC#2 (valid data accepted) PRE count: 2 (both reusable/new identified) Action count: 5 Expected result count: 3 Executable: YES (dev/stg)

>

3. Precondition Reconciliation

For each PRE in TSD:

```
┌─ REUSE: Precondition exists
|  ├─ File: domain/web/pm/factories/preconditions.factory.ts
|  ├─ Function: preconditionManagerUser()
|  ├─ Verify: Executable in dev/stg? YES
|  └─ Ref: Link in TC "PRE#1"
|
├─ EXTEND: Precondition exists but needs expansion
|  ├─ Current: preconditionUserExists()
|  ├─ Extend: Add manager role assignment
|  └─ Action: Update function + document change
|
└─ NEW: Precondition doesn't exist
   ├─ Name: preconditionFormAvailable()
   ├─ Action: Create new function
   ├─ Verify: Executable (GET /v1/pm/create-form → 200)
   └─ Add to factories/preconditions.factory.ts
```

4. Validation: validator-tsd-sync.py

Before handoff, run:

```
python validator-tsd-sync.py \
  --tsd domain/web/pm/TSD-CREATE-EMPLOYEE.md \
  --tc testcase/web/pm/TC-CREATE-EMPLOYEE-*.md

# Output:
# ✅ TC count matches TSD scenarios: 3/3
# ✅ Given/When/Then all present: 100%
# ✅ PRE documented: 2/2
# ✅ Data from FACTORY: 100%
# ✅ Executability verified: dev/stg
```

5. Handoff Checklist

Before passing to GATE C:

```
[ ] TSD validated (self-check done)?
[ ] TC files created per scenario?
[ ] Given/When/Then complete per TC?
[ ] PRE reconciliation done (REUSE/EXTEND/NEW)?
[ ] Validator passed?
[ ] All TC executable in dev/stg?
[ ] Data varied (FACTORY, not hardcoded)?
[ ] TC references TSD section?
[ ] Status: READY FOR AUTOMATION
```

Output: TC files + PRE files (committed to staffinc-testcase)

GATE C: Automation Workflow (Phase 0-6)

Input: Validated TC + PRE files

Tool: `workflow/XX-*.md` phases

Output: Automated test live on stg/dev

Phase 0: Intake

Input: TC file from GATE B

Output: Automation intake document

Steps:

```
1. Read TC completely
   - Scenario understanding
   - PRE requirements
   - Environment needs

2. Ask environment & data questions
   - Target environment: dev/stg/both?
   - Test account: role, email source?
   - Test data: available, setup needed?
   - Cleanup: delete after test, reuse data?
   - Unique identifiers: email randomization?

3. Verify automation status
   - TC marked: "Ready for automation"?
   - PRE available: all PRE files exist?
   - Dependencies: other TC blocked on this?

4. Create intake document
   - File: domain/web/pm/INTAKE-DF-M1-POS-001.md
   - Content: questions answered, decisions made
   - Status: READY FOR GROUNDING
```

Checklist:

```
[ ] TC read completely?
[ ] Environment questions answered?
[ ] PRE verified available?
[ ] Data setup plan clear?
[ ] Cleanup policy decided?
[ ] No blockers?
```

Phase 1: Grounding

Input: Intake document

Output: Grounding document with citations

Steps:

```
1. Ground Given (preconditions)
   - PRE#1: preconditionManagerUser()
     Location: domain/web/pm/factories/preconditions.factory.ts:23
     Endpoint: POST /v1/admin/seed/user
     Verify: Can execute in dev/stg? YES

   - PRE#2: preconditionFormAvailable()
     Location: domain/web/pm/factories/preconditions.factory.ts:45
     Endpoint: GET /v1/pm/create-form
     Verify: Can execute in dev/stg? YES

2. Ground When (UI selectors & actions)
   - Navigate to /pm/employees/create
     Observable in: kerjaan-client-portal/src/pages/CreateEmployee.tsx
     Selector: [data-testid="create-form"]
     Line: src/pages/CreateEmployee.tsx:42

   - Fill name field
     Selector: [data-testid="name"]
     Codebase location: CreateEmployee.tsx:65 (input element)
     Verify: Can interact in dev/stg? YES

3. Ground Then (API response & DB state)
   - Record created in DB
     API: POST /v1/employees
     Response: 201 + { id, status }
     DB table: employees
     Verify: Can read DB in dev/stg? YES

4. Ground data sources
   - Manager email: faker.internet.email() ✓
   - Employee name: faker.person.fullName() ✓
   - From TC: All values traceable to TC ✓
```

Output: Grounding document

```
# Grounding: DF-M1-POS-001

## Given
- [x] PRE#1: preconditionManagerUser() @ factories/preconditions.factory.ts:23
- [x] PRE#2: preconditionFormAvailable() @ factories/preconditions.factory.ts:45

## When
- [x] Navigate /pm/employees/create → CreateEmployee.tsx:42
- [x] Fill [data-testid="name"] → CreateEmployee.tsx:65
- [x] Fill [data-testid="email"] → CreateEmployee.tsx:68
- [x] Click [data-testid="submit"] → CreateEmployee.tsx:71

## Then
- [x] API POST /v1/employees → endpoint available? YES
- [x] DB: employees table exists? YES
- [x] Response: status field? YES

## Data
- [x] Manager email: faker.internet.email() (varied)
- [x] Employee name: faker.person.fullName() (varied)
- [x] All from TC, traceable

Status: GROUNDED ✓
```

Checklist:

```
[ ] All PRE cited with file:line?
[ ] All selectors cited with file:line?
[ ] All API endpoints verified?
[ ] All data from TC (not memory)?
[ ] Executability verified per layer?
[ ] No hardcoded values?
```

Phase 2: Design

Input: Grounding document

Output: Design document with architecture plan

Steps:

```
1. Design assertion for each Then clause

   Then#1: "Record created"
   - Observable: result.id exists
   - Expected: string (UUID)
   - Assertion: expect(result.id).toBeTruthy()
   - Can fail? YES ✓
   - From TC? YES ✓

   Then#2: "Status is DRAFT"
   - Observable: result.status field
   - Expected: 'DRAFT' (from TC)
   - Assertion: expect(result.status).toBe('DRAFT')
   - Can fail? YES ✓ (if code sets ACTIVE)
   - From TC? YES ✓

   And#1: "Notification sent"
   - Observable: Email in queue
   - Expected: recipient = HR team
   - Assertion: expect(emailQueue).toContainEqual({to: 'hr@...'})
   - Can fail? YES ✓
   - From TC? YES ✓

2. Design architecture (DDD layers)

   SPEC layer (test):
   - Import usecase
   - Call with data
   - Assert result

   USECASE layer (domain):
   - Call API POST /v1/employees
   - Handle response
   - Send notification event

   Layer3 (page):
   - Page object for form
   - fill(), click() methods
   - No business logic

   FACTORY layer:
   - generateEmployee() with varied data
   - preconditionManagerUser()
   - preconditionFormAvailable()

3. Error handling plan
   - PRE fails: throw immediately (not swallow)
```

```
   - API error: throw with message
   - Assertion fail: let error propagate
```

Output: Design document

```
# Design: DF-M1-POS-001

## Assertions (per Then clause)

| TC Clause | Observable | Expected | Assertion | Can Fail? |
|-----------|-----------|----------|-----------|-----------|
| "Record created" | result.id | truthy | expect(result.id).toBeTruthy() | ✅ |
| "Status DRAFT" | result.status | 'DRAFT' | expect(result.status).toBe('DRAFT')
| "Notification sent" | email queue | recipient=HR | expect(emailQueue).toContai

## Architecture (DDD)
```

SPEC (test) ├─ Import CreateEmployeeUsecase ├─ Get data from FACTORY ├─ Call usecase.execute(data) └─ Assert response USECASE ├─ POST /v1/employees ├─ Get response 201 ├─ Emit notification event └─ Return { id, status, ... } Layer3 (CreateEmployeePage) ├─ fill(name, email) ├─ click(submit) ├─ wait for response └─ No business logic FACTORY ├─ generateEmployee() → { name, email } ├─ preconditionManagerUser() ├─ preconditionFormAvailable() └─ All data varied

```
## Error Handling

- PRE: throw immediately
- USECASE API error: throw with context
- Assertion: let fail, don't swallow
```

Checklist:

```
[ ] Assertion per Then clause?
[ ] Can fail? YES for all?
[ ] From TC? YES for all?
[ ] DDD layers clear?
[ ] Error handling per layer?
[ ] No hardcoded values?
```

Phase 3: Mapping

Input: Design document

Output: Layer mapping (file structure plan)

Steps:

```
1. Map to SPEC layer
   - File: domain/web/pm/tests/create-employee.spec.ts
   - Test name: test('@covers DF-M1-POS-001', async () => {...})
   - Import usecase & page & factory

2. Map to USECASE layer
   - File: domain/web/pm/usecases/create-employee.usecase.ts
   - Class: CreateEmployeeUsecase
   - Method: execute(data: EmployeeInput)

3. Map to Layer3 (page)
   - File: domain/web/pm/pages/CreateEmployeePage.ts
   - Methods: fill(), click(), waitForResponse()

4. Map to FACTORY layer
   - File: domain/web/pm/factories/employee.factory.ts
   - Function: generateEmployee()
   - File: domain/web/pm/factories/preconditions.factory.ts
   - Function: preconditionManagerUser(), preconditionFormAvailable()
```

Output: Mapping document

```
# Mapping: DF-M1-POS-001

## SPEC Layer
- File: domain/web/pm/tests/create-employee.spec.ts
- Test: test('@covers DF-M1-POS-001')
- Imports:
  - CreateEmployeeUsecase
  - CreateEmployeePage
  - { generateEmployee }

## USECASE Layer
- File: domain/web/pm/usecases/create-employee.usecase.ts
- Class: CreateEmployeeUsecase
- Method: execute(data)
  - POST /v1/employees
  - Emit event
  - Return result

## Layer3 (PAGE)
- File: domain/web/pm/pages/CreateEmployeePage.ts
- Methods:
  - fill(name, email)
  - click()
  - waitForResponse()

## FACTORY Layer
- File: domain/web/pm/factories/employee.factory.ts
  - generateEmployee()
- File: domain/web/pm/factories/preconditions.factory.ts
  - preconditionManagerUser()
  - preconditionFormAvailable()
```

Phase 4: Implement

Input: Mapping document

Output: Code + AUTOMATION-CHECKLIST

Steps:

```
1. Implement per layer (DDD rules)

   ✅ FACTORY (independent):
      export function generateEmployee(overrides = {}) {
        return {
          name: faker.person.fullName(),
          email: faker.internet.email(),
          ...overrides
        }
      }

   ✅ USECASE (domain logic):
      async execute(data: EmployeeInput) {
        const res = await http.post('/v1/employees', data)
        if (!res.ok()) throw new Error(`API error: ${res.status()}`)
        this.eventBus.emit('employee.created', res.json())
        return res.json()
      }

   ✅ Layer3 (page interactions):
      async fill(name: string, email: string) {
        await page.fill('[data-testid="name"]', name)
        await page.fill('[data-testid="email"]', email)
      }

      async click() {
        await page.click('[data-testid="submit"]')
      }

   ✅ SPEC (test assertions):
      test('@covers DF-M1-POS-001', async () => {
        const data = generateEmployee()
        const result = await usecase.execute(data)

        // Then#1: Record created
        expect(result.id).toBeTruthy()

        // Then#2: Status DRAFT
        expect(result.status).toBe('DRAFT')

        // And#1: Notification
        expect(emailQueue).toContainEqual({to: 'hr@...'})
      })

2. Add code comments
   - @covers <TC-ID>
   - // src: filepath:line (selector, endpoint)
```

```
   - // expect: <TC-ID> Then#n (assertion)

3. Create AUTOMATION-CHECKLIST-DF-M1-POS-001.md
   - Map TC clauses to assertions
   - Check: can fail? from TC? no tautology?
   - Self-review: engineer verify complete

4. Run test + collect evidence
   - Test status: PASSED (not skipped/failed)
   - PRE executed: YES
   - Video: recorded (optional)
```

Output: Code files + AUTOMATION-CHECKLIST

Phase 5: Verify

Input: Code + AUTOMATION-CHECKLIST

Output: Evidence + test result

Steps:

```
1. Run test
   - npm test -- create-employee.spec.ts
   - Result: PASSED ✓

2. Verify PRE executed
   - Log: "PRE: Manager user created" ✓
   - Log: "PRE: Form endpoint available" ✓

3. Verify assertions meaningful
   - Assertion #1: status = DRAFT ✓ (can fail)
   - Assertion #2: id exists ✓ (can fail)
   - Assertion #3: email sent ✓ (can fail)

4. Collect evidence
   - Screenshot: form filled + success message
   - Video: full flow (optional)
   - Logs: PRE, USECASE, assertion output

5. Check AUTOMATION-CHECKLIST
   - All TC clauses covered? YES
   - Red flags scanned? YES (no tautology, swallowed error, hardcoded)
   - Test status: PASSED? YES
```

Output: Verification evidence

```
# Verification: DF-M1-POS-001

✅ Test PASSED
✅ PRE executed (2/2)
✅ All assertions meaningful
✅ AUTOMATION-CHECKLIST complete
✅ Evidence collected

Status: READY FOR PR
```

Phase 6: Pre-Push Review & PR

Input: Code + evidence + checklist

Output: PR created + reviewed

Steps:

```
1. Self-review (engineer)
   - AUTOMATION-CHECKLIST complete? YES
   - Red flags? NO (no tautology, swallowed error, hardcoded)
   - Test PASSED? YES
   - Code comments present? YES (@covers, // expect:, // src:)
   - DDD layers correct? YES

2. Create PR
   - Branch: feature/create-employee-automation
   - Title: "test: automate DF-M1-POS-001 (create employee)"
   - Description:
     * TC ID: DF-M1-POS-001
     * Scenarios: POS-001
     * PRE: 2 (manager user + form endpoint)
     * Assertions: 3 (all can fail)
     * Test status: PASSED
     * AUTOMATION-CHECKLIST: complete

3. Reviewer checks (automation-pr-review skill)

   Layer 1 (static code):
   - No tautology? ✓
   - No swallowed error? ✓
   - No hardcoded value? ✓
   - All assertions from TC? ✓

   Layer 2 (traceability):
   - AUTOMATION-CHECKLIST exist? ✓
   - All clauses covered? ✓
   - Red flags checked? ✓

   Layer 3 (executability):
   - PRE executed? ✓
   - FACTORY valid? ✓
   - USECASE endpoint real? ✓
   - Layer3 selectors found? ✓
   - SPEC assertions passed? ✓

   Decision: APPROVED ✓

4. Merge PR
   - TC marked: automated = true
   - Test added to CI
   - Running on stg/dev daily
```

🎯 Summary: E2E Flow

Gate/ Phase	Input	Output	Key Prevention
GATE A	PRD	TSD + self-check	Requirement clarity, scope, PRE
GATE B	TSD	TC + PRE + validation	Data mapping, executability
GATE C-0	TC	Intake doc	Environment questions early
GATE C-1	Intake	Grounding doc	Executability verified per layer
GATE C-2	Grounding	Design doc	Assertions designed, error handling
GATE C-3	Design	Layer mapping	Architecture clear, DDD rules
GATE C-4	Mapping	Code + checklist	Red flags prevented, traceability
GATE C-5	Code	Evidence + result	Test passing, PRE executed
GATE C-6	Evidence	PR + review	Upstream quality confirmed

✅ Quality Gates (Upstream Prevention)

```
✅ Gate A quality:
   - TSD complete (12 sections)
   - AC from PRD (not invented)
   - Scenarios POS/NEG/EDG covered
   - PRE identified

✅ Gate B quality:
   - TC per scenario
   - Given/When/Then complete
   - PRE reconciled (REUSE/EXTEND/NEW)
   - Data varied (not hardcoded)
   - Validator passed

✅ Gate C quality:
   - Phase 0: Environment clear
   - Phase 1: Everything grounded
   - Phase 2: Assertions designed
   - Phase 3: Architecture clear
   - Phase 4: Red flags prevented
   - Phase 5: Test passing
   - Phase 6: PR reviewed upstream
```

References

- **14-executability-principle.md** — Every layer must be executable
- **15-test-failure-reporting.md** — If test fails, diagnose per layer
- **16-assertion-design.md** — Design assertions, prevent red flags
- **17-traceability-record.md** — AUTOMATION-CHECKLIST per TC
- **skills/tsd-writer** — PRD → TSD
- **skills/testcase-writer** — TSD → TC
- **workflow/** — Phases 0-6

19 — PR Review Prerequisites: Locate TC & Understand Context

Goal: Reviewer MUST read & understand Test Case from staffinc-testcase BEFORE running automation-pr-review skill.

🚨 Critical: Review Process is TC-Centric

```
WRONG:
PR received → Run skill automation-pr-review → Review
           ❌ Don't understand what TC expects

RIGHT:
PR received
  ↓
Find TC in staffinc-testcase
  ↓
Read TC: Given/When/Then
  ↓
Understand requirements
  ↓
Run skill automation-pr-review
  ↓
Review with TC context ✓
```

Step 1: Find TC ID from PR

From PR description, extract TC ID:

```
PR Title: "test: automate DF-M1-POS-001 (create employee)"

Look for:
- TC ID: DF-M1-POS-001
- Reference: @covers DF-M1-POS-001
- AUTOMATION-CHECKLIST: AUTOMATION-CHECKLIST-DF-M1-POS-001.md
```

Step 2: Locate TC in staffinc-testcase Repository

Path format:

```
testcase/[domain]/[module]/TC-<FEATURE>-<SCENARIO>.md
```

Examples:

```
testcase/web/pm/TC-CREATE-EMPLOYEE-POS-001.md
testcase/api/auth/TC-LOGIN-POS-001.md
testcase/mobile/dashboard/TC-VIEW-METRICS-NEG-001.md
```

How to Find:

Option 1: Search by TC ID

```
cd staffinc-testcase
grep -r "TC-ID: DF-M1-POS-001" testcase/
grep -r "@covers DF-M1-POS-001" testcase/

# Result:
# testcase/web/pm/TC-CREATE-EMPLOYEE-POS-001.md:TC ID: DF-M1-POS-001
```

Option 2: Browse directory structure

```
staffinc-testcase/
├─ testcase/
|  ├─ web/
|  |  ├─ pm/
|  |  |  ├─ TC-CREATE-EMPLOYEE-POS-001.md
|  |  |  ├─ TC-CREATE-EMPLOYEE-NEG-001.md
|  |  |  └─ TC-CREATE-EMPLOYEE-EDG-001.md
|  |  └─ auth/
|  |     └─ TC-LOGIN-POS-001.md
|  ├─ api/
|  ├─ mobile/
```

Option 3: Check PR description

```
PR mentions:
"Automate TC: testcase/web/pm/TC-CREATE-EMPLOYEE-POS-001.md"
```

Step 3: Read TC Completely

Open TC file and read **all sections**:

Template: What to Read

```
# Test Case: DF-M1-POS-001

## Given (Preconditions)
- What MUST be true before user action
- Example: Manager user exists, form endpoint available
- VERIFY: Can be set up in dev/stg? YES

## When (User Actions)
- Observable sequence of actions
- Example: Navigate → Fill form → Submit
- VERIFY: Can be executed in dev/stg? YES

## Then/And (Expected Results)
- What should be true after actions
- Example: Record created, status = DRAFT, notification sent
- COUNT: How many assertions? (3 in example)

## Preconditions (PRE)
- Specific setup required
- Example: preconditionManagerUser()
- VERIFY: PRE files exist? YES

## Data & State
- Test data sources
- Example: faker.internet.email() (varied, not hardcoded)

## Execution Environment
- Where TC runs: dev/stg/prod?
```

Checklist: Understanding TC

Before moving to review PR, verify you understand:

```
[ ] TC ID clear? (DF-M1-POS-001)
[ ] Feature clear? (Create employee)
[ ] Scenario type? (POS = positive / NEG = negative / EDG = edge case)
[ ] Given: What must be true before test?
[ ] When: What actions does user do? (sequence clear?)
[ ] Then: What should happen? (How many expected results?)
[ ] And: Additional expected results?
[ ] PRE: What preconditions needed? (count)
[ ] Data: From FACTORY or hardcoded? (MUST be FACTORY)
[ ] Executable: Can this run in dev/stg? YES
[ ] All values in Then from TC? YES (not code assumption)
```

Step 4: Map PR to TC

Before running automation-pr-review skill:

Verify PR Matches TC

```
TC Expected:
├─ Given #1: Manager user exists
├─ Given #2: Form endpoint available
├─ When #1: Navigate to /pm/employees/create
├─ When #2: Fill name, email
├─ When #3: Click submit
├─ Then #1: Record created (status DRAFT)
├─ Then #2: Notification sent
└─ PRE: 2 preconditions

PR Code Should Have:
├─ test('@covers DF-M1-POS-001')
├─ preconditionManagerUser() called?
├─ preconditionFormAvailable() called?
├─ page.navigate() to /pm/employees/create?
├─ page.fill() for name, email?
├─ page.click() submit?
├─ expect(result.status).toBe('DRAFT')?
├─ expect(emailQueue).toContainEqual(...)?
└─ AUTOMATION-CHECKLIST-DF-M1-POS-001.md exists?
```

What Should Match:

```
✅ Test name has @covers TC-ID
✅ All Given (PRE) are called
✅ All When (actions) are in code
✅ All Then (assertions) exist and match TC
✅ Data from FACTORY, not hardcoded
✅ AUTOMATION-CHECKLIST complete
❌ Missing any TC clause = REQUEST_CHANGES
```

Step 5: Run automation-pr-review Skill

NOW you're ready. Run skill with TC context:

```
/automation-pr-review \
  --pr-url https://github.com/sulastambunan/staffinc-automation/pull/123 \
  --tc-id DF-M1-POS-001 \
  --tc-file testcase/web/pm/TC-CREATE-EMPLOYEE-POS-001.md
```

Or in chat:

```
@automation-pr-review

PR: #123 (test: automate DF-M1-POS-001)
TC: testcase/web/pm/TC-CREATE-EMPLOYEE-POS-001.md

Context:
- Feature: Create employee
- Scenario: POS-001 (create with valid data)
- PRE: 2 (manager user + form endpoint)
- Assertions: 3 (id, status, notification)

Run review with TC context.
```

Pre-Review Checklist for Reviewer

Before Running Skill:

```
TC Location & Understanding:
[ ] TC ID extracted from PR? (DF-M1-POS-001)
[ ] TC file located in staffinc-testcase? (path known?)
[ ] TC read completely? (all sections understood?)
[ ] Given/When/Then understood? (requirements clear?)
[ ] Preconditions identified? (count: 2, 3, N?)
[ ] Data strategy understood? (FACTORY or hardcoded?)
[ ] Execution environment understood? (dev/stg/both?)

Context Verification:
[ ] PR title mentions TC-ID? YES
[ ] PR description has TC reference? YES
[ ] AUTOMATION-CHECKLIST mentioned? YES
[ ] TC file exists in staffinc-testcase? YES (verified by grep)
[ ] TC matches feature branch? (feature/create-employee matches TC)

Ready for Review?
[ ] Understand exactly what TC expects? YES
[ ] Can trace PR code back to TC clauses? YES (ready)
[ ] Know what red flags to look for? YES (tautology, swallowed error)
[ ] Know what quality gates apply? YES (3 layers)

→ PROCEED TO SKILL
```

Example: Review Flow

Scenario: PR #123 arrives

```
1. Read PR title: "test: automate DF-M1-POS-001 (create employee)"
   Extract: TC ID = DF-M1-POS-001

2. Search in staffinc-testcase:
   $ grep -r "DF-M1-POS-001" testcase/
   → testcase/web/pm/TC-CREATE-EMPLOYEE-POS-001.md

3. Read TC file completely:
   - Given: Manager user exists, form available
   - When: Navigate → Fill → Submit
   - Then: Record created (status DRAFT), notification sent
   - PRE: preconditionManagerUser(), preconditionFormAvailable()
   - Data: faker.internet.email() (not hardcoded)
   - Executable: YES (dev/stg available)

4. Understand expectations:
   - Need 2 assertions minimum (status + notification)
   - Need 2 PRE calls
   - Data must be from FACTORY
   - No hardcoded values

5. Review PR code mentally:
   - ✓ @covers DF-M1-POS-001 present
   - ✓ PRE called: preconditionManagerUser()
   - ✓ PRE called: preconditionFormAvailable()
   - ✓ Actions match TC: navigate, fill, click
   - ✓ Assertions match TC: status DRAFT, notification sent
   - ✓ Data from FACTORY: generateEmployee()
   - ✓ AUTOMATION-CHECKLIST exists

6. Run skill with context:
   /automation-pr-review --tc-id DF-M1-POS-001

7. Skill output aligns with TC understanding:
   - Layer 1: Static code looks good
   - Layer 2: Traceability complete
   - Layer 3: (if executed) All pass

   → APPROVED
```

Red Flags When TC Not Understood

```
❌ "Test looks OK" (without reading TC)
   → WRONG: You don't know what test should do

❌ "Assertion looks reasonable" (assuming from code)
   → WRONG: Must match TC expected value, not code

❌ "Data looks varied" (without checking FACTORY)
   → WRONG: Must verify FACTORY, not just appearance

❌ "PRE not needed" (without reading Given)
   → WRONG: TC Given specifies exactly what PRE needed

✅ "Test matches DF-M1-POS-001 Given/When/Then"
   → RIGHT: You've read and understood TC
```

Summary: Reviewer Workflow

Step	Action	Tool	Output
1	Extract TC ID from PR	Manual read PR	TC-ID: DF-M1-POS-001
2	Locate TC in staffinc-testcase	grep/browse	File path found
3	Read TC completely	Manual read TC	Understanding: Given/When/Then clear
4		Manual trace PR→TC	

Step	Action	Tool	Output
	Verify PR matches TC		Checklist: all clauses covered?
5	Run automation-pr-review skill	/automation-pr-review	Review findings
6	Make decision	APPROVED/ REQUEST_CHANGES	Decision with TC context

---

References

- **18-e2e-prd-to-automation.md** — TC created from TSD in GATE B
- **skills/automation-pr-review/SKILL.md** — Review skill (run after TC understanding)
- **17-traceability-record.md** — AUTOMATION-CHECKLIST verifies TC → code mapping
- **staffinc-testcase repo** — Source of truth for test cases

---

Key Principle

> 🚀 SKILL automation-pr-review is 80% effective only if reviewer has 100% understanding of TC first.
>
> Reviewer without TC understanding = Flying blind.
> Reviewer with TC understanding = Can spot every gap.

Always read TC first. Always.

20 — Fix PR from Feedback: Fixing Automation After PR Review Comments

Goal: When PR review finds issues, systematically fix based on feedback and re-verify before merging.

🔄 Scenario: PR Review → REQUEST_CHANGES → Fix → Merge

```
PR Submitted (automation engineer)
  ↓
PR Review Runs (lead/QA with automation-pr-review skill)
  ↓
Review Result: REQUEST_CHANGES ⚠️
  ├─ Finding 1: Missing assertion for TC clause
  ├─ Finding 2: Hardcoded value in FACTORY
  ├─ Finding 3: Swallowed error in USECASE
  └─ Finding 4: Precondition not executed
  ↓
Engineer Fixes Issues
  ├─ Fix assertion
  ├─ Remove hardcoded
  ├─ Propagate error
  ├─ Execute PRE
  └─ Re-run test
  ↓
Engineer Re-checks AUTOMATION-CHECKLIST
  ├─ All TC clauses covered? ✓
  ├─ Red flags scanned? ✓
  ├─ Test PASSED? ✓
  └─ Status: Ready for re-review
  ↓
Engineer Pushes Fix Commit
  ↓
Lead Re-reviews
  ↓
Decision: APPROVED → Merge
```

Step 1: Read PR Review Feedback

What you receive:

```
PR Review Result: REQUEST_CHANGES

Finding #1: MISSING ASSERTION
  Layer: SPEC
  Line: 26
  Message: "TC Then#1 'Status DRAFT' has no assertion"
  Reference: docs/shared/16-assertion-design.md
  Action: Add assertion for status == DRAFT

Finding #2: HARDCODED VALUE
  Layer: FACTORY
  Line: 45
  Message: "Email hardcoded 'test@test.co', must use faker"
  Reference: docs/shared/16-assertion-design.md#Rule-3
  Action: Use faker.internet.email()

Finding #3: SWALLOWED ERROR
  Layer: USECASE
  Line: 78
  Message: ".catch(() => false) hides error"
  Reference: docs/shared/16-assertion-design.md#Rule-2
  Action: Let error propagate with context

Finding #4: PRECONDITION NOT EXECUTED
  Layer: SPEC
  Line: 12
  Message: "PRE called but no assertion verify success"
  Reference: docs/shared/14-executability-principle.md
  Action: Add expect(preResult.id).toBeTruthy()
```

Step 2: Categorize Issues by Layer

Group findings by DDD layer:

FACTORY Issues (Data generation)

```
Finding #2: Hardcoded email
  File: domain/web/pm/factories/employee-data.factory.ts:45
  Current: const email = 'test@test.co'
  Fix: const email = faker.internet.email()
  Reason: Data must be varied, not hardcoded
  Reference: docs/shared/16-assertion-design.md#Rule-3
```

USECASE Issues (Business logic)

```
Finding #3: Swallowed error
  File: domain/web/pm/usecases/create-employee.usecase.ts:78
  Current: .catch(() => false)
  Fix: .catch(e => { throw new Error(`Create failed: ${e.message}`) })
  Reason: Error must propagate, not hide
  Reference: docs/shared/16-assertion-design.md#Rule-2
```

SPEC Issues (Test assertions)

```
Finding #1: Missing assertion
  File: domain/web/pm/tests/create-employee.spec.ts:26
  Missing: expect(result.status).toBe('DRAFT')
  TC Clause: Then#1 "Status must be DRAFT"
  Reason: Every TC clause must have corresponding assertion
  Reference: docs/shared/16-assertion-design.md#Rule-1

Finding #4: Precondition not verified
  File: domain/web/pm/tests/create-employee.spec.ts:12
  Missing: expect(preResult.id).toBeTruthy()
  Reason: Precondition execution must be asserted
  Reference: docs/shared/14-executability-principle.md
```

Step 3: Fix Each Issue

Fix #1: Add Missing Assertion (SPEC)

Before:

```
test('@covers DF-M1-POS-001', async () => {
  const data = generateEmployee()
  const result = await usecase.execute(data)

  expect(result.id).toBeTruthy()
  // Missing: expect(result.status).toBe('DRAFT')
})
```

After:

```
test('@covers DF-M1-POS-001', async () => {
  const data = generateEmployee()
  const result = await usecase.execute(data)

  // // expect: DF-M1-POS-001 Then#1
  expect(result.status).toBe('DRAFT')

  // // expect: DF-M1-POS-001 Then#2
  expect(result.id).toBeTruthy()
})
```

Commit message:

```
test: add missing assertion for DF-M1-POS-001 Then#1 (status = DRAFT)

- Added expect(result.status).toBe('DRAFT')
- Comment: // expect: DF-M1-POS-001 Then#1
- From TC: staffinc-testcase/testcase/web/pm/DF-M1-POS-001.md
- Fixes: PR review finding #1
```

Fix #2: Remove Hardcoded Value (FACTORY)

Before:

```
export function generateEmployee(overrides = {}) {
  return {
    name: faker.person.fullName(),
    email: 'test@test.co',  // ❌ Hardcoded
    phone: faker.phone.number(),
    ...overrides
  }
}
```

After:

```
export function generateEmployee(overrides = {}) {
  return {
    name: faker.person.fullName(),
    email: faker.internet.email(),  // ✅ Varied per run
    phone: faker.phone.number(),
    ...overrides
  }
}
```

Commit message:

```
refactor: replace hardcoded email with faker.internet.email()

- Changed: email from 'test@test.co' to faker.internet.email()
- Reason: Data must be varied, not hardcoded (red flag prevention)
- Reference: docs/shared/16-assertion-design.md#Rule-3
- Fixes: PR review finding #2
```

Fix #3: Propagate Error (USECASE)

Before:

```
export class CreateEmployeeUsecase {
  async execute(data: EmployeeInput) {
    try {
      const res = await http.post('/v1/employees', data)
      return res.json()
    } catch (e) {
      console.log(e)  // ❌ Swallowed error
      return { success: false }
    }
  }
}
```

After:

```
export class CreateEmployeeUsecase {
  async execute(data: EmployeeInput) {
    try {
      const res = await http.post('/v1/employees', data)
      if (!res.ok()) {
        throw new Error(`API error: ${res.status()}`)
      }
      return res.json()
    } catch (e) {
      // ✅ Error propagates with context
      throw new Error(`Create failed: ${e.message}`)
    }
  }
}
```

Commit message:

```
fix: propagate error instead of swallowing in create employee usecase

- Removed: console.log() + silent return
- Added: Explicit error with context message
- Changed: .catch() now throws instead of returns false
- Reason: Errors must propagate for visibility (red flag prevention)
- Reference: docs/shared/16-assertion-design.md#Rule-2
- Fixes: PR review finding #3
```

Fix #4: Verify Precondition Execution (SPEC)

Before:

```
test('@covers DF-M1-POS-001', async () => {
  const user = await preconditionManagerUser()  // PRE called but not verified

  const data = generateEmployee()
  const result = await usecase.execute(data)

  expect(result.id).toBeTruthy()
})
```

After:

```
test('@covers DF-M1-POS-001', async () => {
  // PRE must execute successfully (asserted)
  const user = await preconditionManagerUser()
  expect(user.id).toBeTruthy()  // ✅ Verify PRE worked
  expect(user.role).toBe('manager')

  const data = generateEmployee()
  const result = await usecase.execute(data)

  expect(result.id).toBeTruthy()
})
```

Commit message:

```
test: add precondition execution verification for DF-M1-POS-001

- Added: expect(user.id).toBeTruthy() after PRE
- Added: expect(user.role).toBe('manager')
- Reason: Precondition must be verified before scenario (executability)
- Reference: docs/shared/14-executability-principle.md
- Fixes: PR review finding #4
```

Step 4: Update AUTOMATION-CHECKLIST

After fixes, update the AUTOMATION-CHECKLIST-.md:

```
# Automation Checklist: DF-M1-POS-001

## Fixes Applied (from PR review)

- [x] Fix #1: Added assertion for status = DRAFT (line 26)
- [x] Fix #2: Removed hardcoded email, use faker (employee.factory.ts:45)
- [x] Fix #3: Propagate error in usecase (create-employee.usecase.ts:78)
- [x] Fix #4: Added PRE verification (line 12)

## Updated Assertions

### Then#1: Status = DRAFT
- [x] Assertion: expect(result.status).toBe('DRAFT')
- [x] Location: create-employee.spec.ts:26
- [x] Comment: // expect: DF-M1-POS-001 Then#1
- [x] Can fail? YES ✓
- [x] From TC? YES ✓
- [x] Fixed: Was missing, now added ✓

### PRE Verification
- [x] Assertion: expect(user.id).toBeTruthy()
- [x] Location: create-employee.spec.ts:12
- [x] Can fail? YES ✓
- [x] Fixed: Was not verified, now explicit ✓

## Status

- Red flags checked: 0 ✓
- All TC clauses: 3/3 ✓
- Test result: PASSED ✓
- Ready for re-review: YES ✓

Updated by: [engineer name]
Date: [YYYY-MM-DD]
PR Review feedback incorporated: ALL ✓
```

Step 5: Run Test Again

Verify all fixes work:

```
# Run test
npm test -- create-employee.spec.ts

# Expected output:
# ✓ Create employee with valid data -> created (42.3s)
# 1 passed

# If FAILED:
# - Read error message
# - Identify which fix didn't work
# - Debug + re-run
# - Update AUTOMATION-CHECKLIST with new finding
```

Step 6: Create Fix Commit

Bundle all fixes in one commit:

```
# Stage all changed files
git add domain/web/pm/tests/create-employee.spec.ts
git add domain/web/pm/usecases/create-employee.usecase.ts
git add domain/web/pm/factories/employee-data.factory.ts
git add domain/web/pm/AUTOMATION-CHECKLIST-DF-M1-POS-001.md

# Commit with clear message referencing PR review
git commit -m "fix: address PR review findings for DF-M1-POS-001

- Fix #1: Add assertion for status = DRAFT (missing Then#1)
- Fix #2: Replace hardcoded email with faker.internet.email()
- Fix #3: Propagate error instead of swallowing in usecase
- Fix #4: Verify precondition execution in test
- Updated: AUTOMATION-CHECKLIST-DF-M1-POS-001.md with fixes

All PR review findings addressed. Test passing. Ready for re-review.

Reference: https://github.com/.../pull/123/files#r123456789
"
```

Step 7: Push & Re-request Review

```
# Push fix commit to same PR branch
git push origin feature/create-employee-automation

# GitHub will automatically:
# - Show new commit in PR
# - Mark previous review as stale
# - Ready for lead to re-review
```

Comment on PR:

```
@lead-reviewer

All findings from PR review #1 addressed:

✅ Fix #1: Added assertion for status = DRAFT
   - Commit: abc1234
   - File: create-employee.spec.ts:26

✅ Fix #2: Replaced hardcoded email with faker
   - Commit: abc1234
   - File: employee-data.factory.ts:45

✅ Fix #3: Propagated error in usecase
   - Commit: abc1234
   - File: create-employee.usecase.ts:78

✅ Fix #4: Added PRE verification
   - Commit: abc1234
   - File: create-employee.spec.ts:12

Test re-run: PASSED ✓
AUTOMATION-CHECKLIST updated: YES ✓

Ready for re-review.
```

Step 8: Lead Re-reviews

Lead reviewer:

1. ✅ Reads new commit
2. ✅ Verifies each fix addresses finding
3. ✅ Runs automation-pr-review skill again (optional, should pass)
4. ✅ Decision: APPROVED
5. ✅ Merges PR

📋 Checklist: Fixing PR from Feedback

```
Issues Received:
[ ] All findings read and understood?
[ ] Categorized by layer (FACTORY/USECASE/SPEC)?
[ ] Priority determined (blocker vs nice-to-have)?

Each Fix:
[ ] Issue understood (what + why)?
[ ] Root cause fixed (not just symptom)?
[ ] Code tested (test still passes)?
[ ] Comment added (reference to finding)?
[ ] AUTOMATION-CHECKLIST updated?

Before Re-review:
[ ] All findings addressed?
[ ] Test PASSED?
[ ] No new issues introduced?
[ ] Commit message clear?
[ ] Push to same PR branch?
[ ] Comment on PR requesting re-review?
```

🎯 Key Points

```
✅ Fixes are UPSTREAM PREVENTION applied DOWNSTREAM
   - Should have been caught at Phase 4 (Implement)
   - But caught at Phase 6 (PR Review)
   - Still better than production bug!

✅ Each fix must be TRACEABLE to PR finding
   - Commit message references finding
   - Code comment references TC + finding
   - AUTOMATION-CHECKLIST updated

✅ No band-aids
   - Fix ROOT cause, not symptom
   - Don't just make test pass
   - Make test meaningful per TC

✅ Re-run test AFTER each fix
   - Don't assume fix works
   - Verify with actual execution
```

References

- **16-assertion-design.md** — Why each fix matters (rules)
- **14-executability-principle.md** — Why PRE must be verified
- **17-traceability-record.md** — AUTOMATION-CHECKLIST update
- **skills/automation-pr-review/SKILL.md** — Where findings come from

---

21 — Environment Confirmation: Dev → STG → Regression Ready

Goal: After automation passes in dev, MUST verify it works in staging for regression testing before marking as complete.

🚀 Critical: Automation NOT "Done" Until STG-Ready

```
WRONG:
  Test passes in dev
  → "Automation complete!"
  → Code merged
  → Deployed to stg
  → Test FAILS in stg ⚠️
  → Emergency fix needed
  → Delayed regression testing

RIGHT:
  Test passes in dev ✓
    ↓
  Setup confirmed in stg ✓
    ↓
  Test passes in stg ✓
    ↓
  Regression ready ✓
    ↓
  Code merged
  → "Automation complete AND regression-ready!"
```

Phase 5.5: Environment Confirmation (NEW PHASE)

When: After Phase 5 Verify (test passing in dev)

Before: Phase 6 (PR Review)

Responsibility: Automation engineer + QA lead

Step 1: Identify Environment Dependencies

Before running in stg, document what's needed:

Checklist: Environment Requirements

```
Test Data:
  [ ] Manager user needed? (role, email domain)
  [ ] Employee sample data needed? (count, state)
  [ ] Initial state in DB? (tables pre-populated?)
  [ ] Test data cleanup policy? (delete after / reuse)

Infrastructure:
  [ ] API endpoint same as dev?
  [ ] Database accessible from test runner?
  [ ] Queue system available? (async processing)
  [ ] Email service available? (send/capture)
  [ ] File storage available? (S3, local)

Configuration:
  [ ] Environment variables different? (API_URL, DB_HOST)
  [ ] Feature flags different in stg? (enabled/disabled)
  [ ] Rate limiting applied in stg? (might fail tests)
  [ ] Network restrictions? (VPN needed, firewall)

Data Parity:
  [ ] Schema same as dev?
  [ ] Default values same?
  [ ] Constraints same? (unique constraints, FK)
  [ ] Sequences reset? (ID generation)

Cleanup:
  [ ] Previous test data present?
  [ ] Need data isolation strategy?
  [ ] Transaction rollback? or delete records?
  [ ] Orphaned data from failed runs?
```

Step 2: Prepare STG Environment

Setup test data and state in staging:

Example: Create Employee Test in STG

```
# 1. Verify connectivity to stg
curl -X GET https://stg.api.hcm.internal/health
# Expected: 200 OK

# 2. Verify database access
mysql -h stg-db.internal -u test_user -p << EOF
SELECT VERSION();
SELECT COUNT(*) FROM users;
EOF

# 3. Create manager user (precondition) in stg
curl -X POST https://stg.api.hcm.internal/v1/admin/seed/user \
  -H "Authorization: Bearer stg-test-token" \
  -H "Content-Type: application/json" \
  -d '{
    "role": "manager",
    "email": "mgr-stg-test-'$(date +%s)'@test.hcm.internal",
    "name": "STG Test Manager"
  }'
# Expected: 201 Created

# 4. Verify precondition worked
mysql -h stg-db.internal -u test_user -p << EOF
SELECT id, role, email FROM users WHERE role='manager'
  AND email LIKE 'mgr-stg-test-%';
EOF
# Expected: 1 row returned

# 5. Document state for test
cat > domain/web/pm/STG-ENVIRONMENT-SETUP.md << 'EOF'
# STG Environment Setup: DF-M1-POS-001

## Verified Date: 2026-01-20 10:30 UTC

### Connectivity
- [x] API endpoint: stg.api.hcm.internal (200 OK)
- [x] Database: stg-db.internal:3306 (accessible)
- [x] Queue: stg-queue.internal:5672 (accessible)

### Preconditions
- [x] Manager user created
  - ID: user_123
  - Email: mgr-stg-test-1234567890@test.hcm.internal
  - Verified in DB: YES

### Data State
- [x] users table: 15 records (clean)
```

```
- [x] employees table: 0 records (clean)
- [x] audit_log: ready

### Configuration
- [x] API_URL=https://stg.api.hcm.internal
- [x] DB_HOST=stg-db.internal
- [x] FEATURE_FLAG_NEW_FORM=enabled
- [x] RATE_LIMIT=1000/min (should not affect test)

### Ready for Test
Status: YES ✓
Confirmed by: automation_engineer
Date: 2026-01-20 10:30 UTC
EOF
```

Step 3: Run Test in STG

Execute test in staging environment:

```
# 1. Set environment to staging
export ENVIRONMENT=stg
export API_URL=https://stg.api.hcm.internal
export DB_HOST=stg-db.internal
export TEST_ACCOUNT_EMAIL=mgr-stg-test-1234567890@test.hcm.internal

# 2. Run test
npm test -- create-employee.spec.ts --project=chromium

# Expected output:
# ✓ Create employee with valid data -> created (45.2s) [STG]

# If FAILED:
# - Capture error + screenshot
# - Check: Is it test code issue or stg environment issue?
# - Investigate root cause
# - Fix and re-run

# 3. Document result
cat > domain/web/pm/STG-TEST-RUN-RESULT.md << 'EOF'
# STG Test Run: DF-M1-POS-001

## Test Execution: 2026-01-20 10:45 UTC

### Environment
- Environment: STG
- API: stg.api.hcm.internal
- Database: stg-db.internal

### Test Result
Status: PASSED ✓
Duration: 45.2s
Platform: Chromium
Test: @covers DF-M1-POS-001 — Create employee with valid data

### Assertions
- [x] PRE executed: manager user created
- [x] Action: Navigate → Fill → Submit
- [x] Then#1: status = DRAFT
- [x] Then#2: id exists
- [x] And#1: createdBy = current user

### Evidence
- Video: outputs/evidence/2026-01-20/DF-M1-POS-001-stg.webm
- Screenshot: outputs/evidence/2026-01-20/success.png
- Logs: outputs/logs/2026-01-20/DF-M1-POS-001-stg.log
```

```
### Data Created
- Employee ID: emp_456
- Status: DRAFT
- Created in DB: YES ✓

### Cleanup
- Test data deleted: YES
- Rollback: Complete ✓
- State after test: Clean ✓

Confirmed by: automation_engineer
Date: 2026-01-20 10:45 UTC
EOF
```

Step 4: Verify Regression Readiness

Confirm automation is ready for regression testing:

Checklist: Regression Ready

```
Code Quality:
  [ ] Test passing in dev? YES
  [ ] Test passing in stg? YES
  [ ] No hardcoded values? YES
  [ ] No environment-specific logic? YES
  [ ] Assertions meaningful? YES

Environment Compatibility:
  [ ] Works in dev? YES
  [ ] Works in stg? YES
  [ ] Precondition executable in stg? YES
  [ ] Data isolation working? YES
  [ ] Cleanup working? YES

Documentation:
  [ ] AUTOMATION-CHECKLIST complete? YES
  [ ] STG-ENVIRONMENT-SETUP documented? YES
  [ ] STG-TEST-RUN-RESULT documented? YES
  [ ] Precondition documented? YES
  [ ] Data cleanup documented? YES

Maintenance:
  [ ] Instructions for running: Clear? YES
  [ ] Troubleshooting guide: Present? YES
  [ ] Known issues: None? YES
  [ ] Flakiness: None observed? YES

Ready for Regression:
  [ ] Can run daily? YES
  [ ] Can run against new code changes? YES
  [ ] Can scale to multiple runs? YES
  [ ] Will catch regressions? YES
```

Step 5: Document for QA/Regression Team

Create runbook for regression testing:

```
# Regression Test Runbook: DF-M1-POS-001

## Test Overview
- **ID:** DF-M1-POS-001
- **Feature:** Create Employee
- **Scenario:** Create employee with valid data
- **Test Status:** Automated ✓
- **Environment:** STG (primary) + DEV (development)
- **Regression Frequency:** Daily (every commit)

## How to Run

### Prerequisites
```bash
export ENVIRONMENT=stg
export API_URL=https://stg.api.hcm.internal
export DB_HOST=stg-db.internal
```

Command

```
npm test -- create-employee.spec.ts --project=chromium

# Or with filtering:
npm test -- --grep "@covers DF-M1-POS-001"
```

Expected Duration

- 45-50 seconds (includes PRE setup + test + cleanup)

What It Tests

1. **Precondition:** Manager user exists
2. **Action:** Navigate to create form → Fill → Submit
3. **Expected Results:**
4. Employee record created
5. Status = DRAFT (not ACTIVE)
6. Employee ID assigned
```

7. Created by current user

Success Criteria

- [x] Test passes (green ✓)
- [x] No network errors
- [x] No database errors
- [x] Data cleanup complete (no orphaned records)

Failure Diagnosis

If Test Fails

1. **Check precondition:**
2. Is manager user API responding?
3. Can test access database? `bash curl https://stg.api.hcm.internal/health mysql -h stg-db.internal -u test_user -p -e "SELECT 1"`
4. **Check UI changes:**
5. Did selectors change?
6. Is form still at /employees/create?
7. Check: git log --oneline domain/web/pm/
8. **Check feature regression:**
9. Is status field still created?
10. Is it still set to DRAFT on creation?
11. Check: Employee creation logic
12. **Capture evidence:**
13. Screenshot (included automatically)

14. Video (included automatically)
15. Logs: ./outputs/logs/

Common Failures

- ❌ "Selector not found": UI changed, update grounding
- ❌ "HTTP 500": API error, check server logs
- ❌ "Timeout": Network issue, check connectivity
- ❌ "Status = ACTIVE": Code regression, check logic

Maintenance

When to Update Test

- [ ] UI selector changed → Update PAGE layer
- [ ] API response format changed → Update USECASE layer
- [ ] Expected value changed → Update TC + SPEC layer
- [ ] Precondition changed → Update FACTORY layer

Who to Contact

- **Automation Issues:** @automation-team
- **Feature Regression:** @pm-team
- **Infrastructure Issues:** @devops-team

History

- **Created:** 2026-01-15
- **Last Run:** 2026-01-20 10:45 UTC
- **Status:** Passing consistently

- **Flakiness:** None observed

```
---

## Step 6: Confirm with QA Lead

**Get approval before marking complete:**

```markdown
QA Lead Verification Checklist:

- [ ] Test passes in STG? YES/NO
- [ ] Preconditions working? YES/NO
- [ ] Data cleanup complete? YES/NO
- [ ] No flakiness observed? YES/NO
- [ ] Documentation clear? YES/NO
- [ ] Runbook ready? YES/NO
- [ ] Ready for daily regression? YES/NO

Approval: ✓ APPROVED FOR REGRESSION
Signed by: [QA Lead Name]
Date: [YYYY-MM-DD HH:MM UTC]

Status: Automation complete + regression-ready
```

Phase 5.5 Complete: Environment Confirmation Done ✓

At this point:

```
✅ Test passes in dev
✅ Test passes in stg
✅ Preconditions verified in stg
✅ Data isolation working
✅ Cleanup working
✅ Documentation complete
✅ Regression runbook ready
✅ QA lead approved

→ READY FOR PHASE 6 (PR Review + Merge)
```
```

What Changes from DEV to STG?

Only configuration, NOT code:

```
❌ WRONG: Different test code for stg
  if (process.env.ENVIRONMENT === 'stg') {
    // different assertions
  }

✅ RIGHT: Same test code, different config
  API_URL=https://stg.api.hcm.internal
  DB_HOST=stg-db.internal
  (test code exactly same)
```

Why This Matters for Regression

```
Regression test runs EVERY commit:
  1. Engineer makes code change
  2. CI runs automation tests
  3. Test runs in STG
  4. If fails: regression detected, PR blocked
  5. Engineer must fix regression
  6. Test passes: no regression, PR approved

For this to work:
  ✅ Test must be reliable in stg
  ✅ No flakiness
  ✅ Preconditions always work
  ✅ Data always gets cleaned up
  ✅ Assertions always meaningful

If ANY of above fails:
  ❌ Regression tests become noisy
  ❌ False positives → engineers ignore results
  ❌ Real regressions slip through
  ❌ QA finds bugs in prod 💥
```

References

- **14-executability-principle.md** — Why "everything executable in stg"
- **05-verify-evidence.md** — Evidence collection in dev
- **skills/automation-pr-review/SKILL.md** → Now can check: "does it work in stg?"
- **regression-testing-strategy.md** (future) — How to run automation daily

---